

LAPORAN PROYEK UTAMA INFORMATIKA
Semester Genap 2025/2026

**PENGEMBANGAN SISTEM PENYIMPANAN IMMUTABLE OBJECT
STORAGE BERBASIS CONTENT-ADDRESSABLE STORAGE DAN
ALGORITMA FAST CONTENT-DEFINED CHUNKING UNTUK
MITIGASI RANSOMWARE**

Dosen Pembimbing: Moh. Ali Romli, S.Kom., M.Kom.



ALFIAN GADING SAPUTRA (5230411121)

**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS & TEKNOLOGI
UNIVERSITAS TEKNOLOGI YOGYAKARTA
YOGYAKARTA
2026**

LEMBAR PENGESAHAN

LAPORAN PROYEK UTAMA INFORMATIKA

Semester Genap 2025/2026

**PENGEMBANGAN SISTEM PENYIMPANAN IMMUTABLE OBJECT
STORAGE BERBASIS CONTENT-ADDRESSABLE STORAGE DAN
ALGORITMA FAST CONTENT-DEFINED CHUNKING UNTUK
MITIGASI RANSOMWARE**

Laporan ini telah disahkan oleh pembimbing
pada tanggal

Dosen Pembimbing

Moh. Ali Romli, S.Kom., M.Kom.

NIK 110221182

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa karena Laporan Proyek Utama Informatika yang berjudul “Pengembangan Sistem Penyimpanan Immutable Object Storage Berbasis Content-Addressable Storage dan Algoritma Fast Content-Defined Chunking untuk Mitigasi Ransomware” dapat diselesaikan dengan baik. Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis menyampaikan terima kasih kepada:

- a. Dr. Bambang Moertono Setiawan, M.M., Akt., CA. selaku Rektor Universitas Teknologi Yogyakarta.
- b. Prof. Dr. Ar. Endy Marlina, S.T., M.T. selaku Dekan Fakultas Sains & Teknologi Universitas Teknologi Yogyakarta.
- c. Dr. Donny Avianto, M.T. selaku Ketua Program Studi Informatika Universitas Teknologi Yogyakarta.
- d. Moh. Ali Romli, S.Kom., M.Kom. selaku Dosen Pembimbing Mata Kuliah Proyek Utama Informatika.
- e. Kedua orang tua, keluarga, dan teman-teman yang telah memberikan doa, semangat, serta dukungan kepada penulis.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat disempurnakan pada tahap berikutnya. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi referensi awal dalam pengembangan sistem penyimpanan cadangan yang aman dan efisien.

Yogyakarta, Mei 2026

Penulis

ABSTRAK

Repositori cadangan merupakan komponen penting dalam proses pemulihan data ketika terjadi serangan *ransomware*. Namun, arsitektur penyimpanan cadangan yang masih bersifat *mutable* dapat memberikan peluang bagi penyerang untuk menghapus atau menimpa data cadangan melalui jalur aplikasi yang telah diretas. Oleh karena itu, perlu dikembangkan sistem penyimpanan cadangan yang mampu menjaga integritas data sekaligus mengurangi pemborosan kapasitas penyimpanan. Penelitian ini mengembangkan prototipe sistem *Immutable Object Storage* berbasis *Content-Addressable Storage* (CAS) dan algoritma *Fast Content-Defined Chunking* (FastCDC) pada lingkungan *single-node*. Sumber data penelitian berasal dari sampel berkas simulasi, meliputi berkas identik, berkas dengan perubahan sebagian, berkas berbeda sepenuhnya, dan skenario unggah tidak selesai. Sistem memproses berkas menjadi *chunk* menggunakan FastCDC, menghitung identitas *chunk* menggunakan *hash* BLAKE3, menyimpan *manifest* objek pada BadgerDB, serta mengelola metadata aplikasi menggunakan PostgreSQL. Pengujian dilakukan melalui skenario unggah, unduh, deduplikasi, *soft delete*, rekonstruksi objek, kegagalan proses unggah, dan simulasi manipulasi melalui lapisan API. Hasil penelitian menunjukkan bahwa prototipe mampu memperlihatkan konsep penyimpanan *immutable*, mendukung efisiensi kapasitas melalui penggunaan ulang *chunk* yang sama, merekonstruksi objek berdasarkan *manifest*, serta membatasi operasi destruktif terhadap *Vault Core* sesuai rancangan sistem.

Kata Kunci: *Immutable Object Storage*, *Content-Addressable Storage*, FastCDC, Deduplikasi, *Ransomware*.

DAFTAR ISI

SAMPUL	i
LEMBAR PENGESAHAN.....	ii
KATA PENGANTAR.....	iii
ABSTRAK	iv
DAFTAR ISI	v
DAFTAR GAMBAR	vii
DAFTAR TABEL	viii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Ruang Lingkup.....	2
1.4 Tujuan dan Manfaat.....	3
1.5 Sistematika	4
BAB II TINJAUAN PUSTAKA DAN TEORI	6
2.1 Tinjauan Pustaka	6
2.2 Teori	13
2.2.1 Immutable Object Storage.....	13
2.2.2 Write-Once-Read-Many (WORM)	13
2.2.3 Content-Addressable Storage (CAS)	14
2.2.4 Content-Defined Chunking	15
2.2.5 Fast Content-Defined Chunking (FastCDC)	16
2.2.6 Kriptografi dan Fungsi Hash	16
2.2.7 BLAKE3	17
2.2.8 Basis Data	18
2.2.9 BadgerDB.....	18
2.2.10 PostgreSQL.....	19
2.2.11 Go (Golang).....	19
2.2.12 React	20
2.2.13 Ransomware.....	20
2.2.14 Flowchart.....	21
2.2.15 Data Flow Diagram	23
2.2.16 Entity Relationship Diagram.....	24
2.2.17 Pengujian Black Box	25

2.2.18	Pengujian Keamanan.....	25
BAB III METODE PENELITIAN		27
3.1	Kerangka Penelitian.....	27
3.2	Data Penelitian.....	29
3.2.1	Sumber Data	29
3.2.2	Mendapatkan Data	30
3.2.3	Waktu Pengumpulan Data	30
3.3	Arsitektur Model.....	31
3.4	Analisis dan Perancangan.....	32
3.4.1	Kebutuhan Fungsional.....	32
3.4.2	Kebutuhan Nonfungsional	34
3.4.3	Perancangan Konseptual dan Fisik	35
BAB IV PRODUK		54
4.1	Hasil	54
4.2	Pembahasan Hasil	55
4.2.1	Proses Penyimpanan dan Deduplikasi Objek.....	55
4.2.2	User Interface	56
4.2.3	Pengujian Sistem.....	57
4.3	Pengembangan Ke Tugas Akhir	61
BAB V SIMPULAN		62
DAFTAR PUSTAKA		63
LAMPIRAN		66

DAFTAR GAMBAR

Gambar 3.1 Kerangka Penelitian.....	Error! Bookmark not defined.
Gambar 3.2 Arsitektur Model Sistem.....	Error! Bookmark not defined.
Gambar 3.3 Entity Relationship Diagram (ERD).....	Error! Bookmark not defined.
Gambar 3.4 Data Flow Diagram (DFD) Level 0.....	Error! Bookmark not defined.
Gambar 3.5 Data Flow Diagram (DFD) Level 1.....	Error! Bookmark not defined.
Gambar 3.6 Data Flow Diagram (DFD) Level 2 Proses 1.....	Error! Bookmark not defined.
Gambar 3.7 Data Flow Diagram (DFD) Level 2 Proses 2.....	Error! Bookmark not defined.
Gambar 3.8 Data Flow Diagram (DFD) Level 2 Proses 3.....	Error! Bookmark not defined.
Gambar 3.9 Data Flow Diagram (DFD) Level 2 Proses 4.....	Error! Bookmark not defined.
Gambar 3.10 Flowchart Validasi Operasi Destruktif terhadap Vault Core.....	Error! Bookmark not defined.
Gambar 3.11 Flowchart Proses Unggah dan Deduplikasi Berkas.....	Error! Bookmark not defined.
Gambar 3.12 Flowchart Proses Rekonstruksi dan Pengunduhan Objek	Error! Bookmark not defined.
Gambar 3.13 Wireframe Halaman Dasbor	Error! Bookmark not defined.
Gambar 3.14 Wireframe Halaman Pratinjau Berkas.....	Error! Bookmark not defined.
Gambar 4.1 Dasbor Sistem.....	56
Gambar 4.2 Rincian Berkas.....	57

DAFTAR TABEL

Tabel 2.1 Sumber Pustaka Primer	11
Tabel 2.2 Simbol Flowchart	21
Tabel 2.3 Simbol Data Flow Diagram.....	23
Tabel 2.4 Simbol Entity Relationship Diagram	24
Tabel 3.1 Sumber Data Penelitian	29
Tabel 3.2 Tabel Pengguna	50
Tabel 3.3 Tabel Direktori	50
Tabel 3.4 Tabel Relasi Direktori	51
Tabel 3.5 Tabel Berkas.....	51
Tabel 4.1 Hasil Pengujian	58

BAB I

PENDAHULUAN

1.1 Latar Belakang

Evolusi serangan *ransomware* tidak lagi sekadar mengenkripsi data operasional, melainkan secara agresif menargetkan repositori cadangan (*backup*) untuk menghilangkan opsi pemulihan korban. Cybersecurity and Infrastructure Security Agency dan Multi-State Information Sharing and Analysis Center menekankan pentingnya menjaga cadangan data secara *offline* karena pelaku *ransomware* umumnya berupaya menemukan, menghapus, atau mengenkripsi *backup* yang masih dapat diakses (Cybersecurity and Infrastructure Security Agency, 2023). CISA juga merekomendasikan agar data cadangan dienkripsi, bersifat immutable, dan mencakup seluruh infrastruktur data organisasi (Cybersecurity and Infrastructure Security Agency., 2024). Sejalan dengan itu, laporan Sophos menunjukkan bahwa 94% organisasi yang terkena *ransomware* mengalami upaya kompromi terhadap *backup*, dengan 57% upaya kompromi *backup* berhasil berdampak pada proses pemulihan korban (Sophos, 2024). Keberhasilan serangan dalam merusak data cadangan dapat disebabkan oleh arsitektur penyimpanan cadangan yang masih bersifat *mutable* atau dapat diubah. Akibatnya, saat penyerang menguasai server aplikasi, mereka menyalahgunakan jalur autentikasi sah untuk menghapus atau menimpa data cadangan secara permanen. Tanpa mekanisme pertahanan yang memisahkan domain kontrol aplikasi dari domain penyimpanan fisik, organisasi kehilangan garis pertahanan terakhir.

Menghadapi ancaman manipulasi repositori cadangan, penelitian yang dilakukan bertujuan membangun arsitektur penyimpanan yang dirancang untuk mencegah penghapusan data melalui jalur aplikasi yang diretas. Salah satu solusi standar untuk mencegah modifikasi data adalah teknologi *Write-Once-Read-Many* (WORM). Walaupun WORM mencegah penghapusan data dalam periode tertentu, penerapan WORM memicu inefisiensi kapasitas penyimpanan. Perubahan kecil pada berkas berukuran besar memaksa sistem WORM konvensional menyimpan ulang seluruh berkas sebagai objek baru, sehingga memicu lonjakan kebutuhan

kapasitas (*data bloat*). Fokus utama penelitian yang dilakukan adalah mengimplementasikan mekanisme efisiensi penyimpanan tanpa mengorbankan integritas data yang telah tersimpan.

Pendekatan untuk menjembatani kebutuhan keamanan dan efisiensi mengarah pada pengembangan sistem *Immutable Object Storage*. Sistem mengintegrasikan *Content-Addressable Storage* (CAS) dengan algoritma *Fast Content-Defined Chunking* (FastCDC). Algoritma FastCDC memecah berkas menjadi potongan-potongan data atau *chunk* berdasarkan isi konten, sehingga sistem hanya menyimpan *chunk* baru melalui mekanisme deduplikasi. Kombinasi CAS, FastCDC dan pemisahan domain kontrol memungkinkan sistem menyimpan data secara *immutable*, sementara deduplikasi tingkat *chunk* membantu meningkatkan efisiensi penggunaan kapasitas penyimpanan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah pada penelitian ini adalah bagaimana sistem *Immutable Object Storage* berbasis *Content-Addressable Storage* (CAS) dan algoritma *Fast Content-Defined Chunking* (FastCDC) dapat diimplementasikan pada lingkungan *single-node* untuk melindungi repositori cadangan dari manipulasi *ransomware* melalui mekanisme *immutability* logis dan pemisahan otoritas penyimpanan.

1.3 Ruang Lingkup

Penelitian pengembangan *Immutable Object Storage* mencakup berbagai hal, sebagai berikut:

- a. Penelitian berfokus pada penerapan *Content-Addressable Storage* (CAS) dengan algoritma *Fast Content-Defined Chunking* (FastCDC) untuk mencapai efisiensi penyimpanan melalui deduplikasi dan *immutability* logis.
- b. Aplikasi web yang dikembangkan hanya berfungsi sebagai antarmuka demonstrasi untuk autentikasi, pengelolaan direktori logis, pengunggahan berkas, penampilan metadata dasar dan metadata keamanan, penandaan berbintang, *soft delete*, pemulihan item, serta pengunduhan kembali objek.

Penelitian tidak berfokus pada evaluasi pengalaman pengguna, kelengkapan fitur manajemen dokumen, *dashboard* analitik, maupun sistem administrasi aplikasi web.

- c. Sistem mencakup mekanisme retensi dan *soft delete* sebagai penandaan status logis pada metadata aplikasi. Mekanisme ini tidak menghapus *chunk* fisik, manifest objek, atau referensi penyimpanan di *Vault Core*. Penelitian tidak mencakup proses penghapusan fisik maupun *garbage collection* (GC).
- d. Penelitian hanya difokuskan pada pengujian di lingkungan *single-node* dengan isolasi berbasis pemisahan hak akses sistem operasi dan komunikasi lokal melalui *Unix Domain Socket* (UDS), sehingga tidak mencakup isolasi fisik jaringan antar-server, *high availability*, maupun replikasi terdistribusi.
- e. Modul pengunduhan data atau *retrieval* mengizinkan lapisan API memiliki akses baca langsung atau *read-only mount* ke direktori penyimpanan fisik, dan belum menggunakan lapisan proksi baca atau *read-proxy*.
- f. Prototipe sistem dikembangkan untuk menguji kemampuan perlindungan repositori cadangan dari manipulasi melalui jalur aplikasi yang diretas, mengukur rasio deduplikasi, serta menguji konsistensi data setelah kegagalan sistem atau *crash consistency*.

1.4 Tujuan dan Manfaat

Tujuan penelitian adalah mengembangkan prototipe sistem *Immutable Object Storage* yang efisien dan aman untuk menjaga integritas data cadangan saat server aplikasi mengalami peretasan. Penelitian juga bertujuan untuk mengimplementasikan algoritma *Fast Content-Defined Chunking* (FastCDC) dalam arsitektur *Content-Addressable Storage* (CAS). Selain itu, penelitian mengevaluasi konsistensi metadata saat terjadi kegagalan sistem dan mengukur efisiensi penyimpanan melalui deduplikasi tingkat *chunk*. Melalui tujuan tersebut, risiko manipulasi data cadangan akibat *ransomware* dan ketidakkonsistenan metadata saat terjadi kegagalan proses penyimpanan dapat diminimalkan.

Manfaat penelitian pengembangan sistem penyimpanan *Immutable Object Storage* untuk mitigasi *ransomware* adalah sebagai berikut:

- a. Memberikan rancangan awal sistem penyimpanan cadangan yang mampu mengurangi peluang manipulasi data melalui pemisahan otoritas antara lapisan aplikasi dan lapisan penyimpanan.
- b. Mengurangi pemborosan kapasitas penyimpanan pada skenario data cadangan melalui penerapan deduplikasi tingkat *chunk* berbasis CAS dan FastCDC.
- c. Menjadi acuan awal bagi pengembang sistem keamanan siber dalam membangun repositori cadangan *immutable* yang tetap mempertimbangkan efisiensi kapasitas penyimpanan.
- d. Menyediakan prototipe demonstratif yang dapat digunakan untuk memperlihatkan alur penyimpanan, deduplikasi, dan rekonstruksi objek kepada pengguna atau penguji.

1.5 Sistematika

Sistematika penulisan laporan Proyek Utama Informatika disusun ke dalam beberapa bab sebagai berikut:

Bab I Pendahuluan

Bab ini menguraikan latar belakang masalah kerentanan data cadangan, rumusan masalah, batasan ruang lingkup sistem, serta tujuan dan manfaat penelitian pengembangan *Immutable Object Storage*.

Bab II Tinjauan Pustaka dan Teori

Bab ini membahas hasil penelitian sebelumnya yang relevan dengan topik mitigasi *ransomware* dan penyimpanan data. Bagian teori membahas konsep dasar yang mendukung penelitian, meliputi *Content-Addressable Storage* (CAS), algoritma *Fast Content-Defined Chunking* (FastCDC), prinsip kriptografi *hashing*, serta mekanisme keamanan sistem penyimpanan WORM.

Bab III Metode Penelitian

Bab ini menjelaskan tahapan penelitian, data penelitian, arsitektur model, serta analisis dan perancangan sistem penyimpanan *Immutable Object*

Storage. Bab ini juga menjelaskan antarmuka web sebagai lapisan demonstrasi yang menghubungkan pengguna dengan *Vault Core*.

Bab IV Produk

Bab ini memaparkan hasil implementasi prototipe sistem *Immutable Object Storage*, pembahasan hasil pengujian terhadap skenario normal dan tidak normal, serta arah pengembangan fitur lanjutan berupa *Concurrent Garbage Collection* dan *Read-Proxy* pada tahap Proyek Profesional dan Tugas Akhir.

Bab V Simpulan

Bab ini membahas kesimpulan dari hasil pengembangan prototipe awal sistem, keterbatasan penelitian, serta arah pengembangan sistem pada tahap berikutnya.

Daftar Pustaka

Daftar pustaka berisi sitasi dari berbagai referensi yang digunakan dalam penyusunan laporan, yang meliputi jurnal ilmiah, prosiding konferensi, dan dokumentasi teknis terkait.

BAB II TINJAUAN PUSTAKA DAN TEORI

2.1 Tinjauan Pustaka

Tinjauan pustaka menguraikan berbagai penelitian terdahulu yang relevan dengan pengembangan arsitektur penyimpanan dan mitigasi *ransomware*. Penulis mengintegrasikan temuan-temuan sebelumnya sebagai landasan teoretis untuk membangun kebaruan rancangan sistem pada penelitian yang dilakukan.

Penelitian Caporaso dkk., (2024) membahas pengembangan sistem berkas *write-once* bernama VaultFS sebagai solusi mitigasi serangan *ransomware*. Peneliti menjadikan sistem berkas tingkat *kernel* pada lingkungan Linux sebagai objek penelitian dan memodifikasinya untuk menolak operasi penulisan ulang. Peneliti mengimplementasikan kebijakan *write-once* pada lapisan VFS (*Virtual File System*). Hasil penelitian menunjukkan bahwa pemisahan domain kontrol pada tingkat sistem berkas merupakan garis pertahanan krusial saat terjadi peretasan. Akun administratif yang diretas tetap tidak dapat memodifikasi data yang terkunci. Penelitian mengangkat permasalahan terkait ketiadaan mekanisme proteksi lapisan sistem berkas yang independen dari jalur autentikasi aplikasi. Keterbatasan penelitian terdahulu terletak pada fokus proteksi tingkat sistem berkas yang belum mempertimbangkan efisiensi kapasitas penyimpanan. Penelitian menutup celah penelitian terdahulu dengan menambahkan lapisan deduplikasi berbasis *Content-Addressable Storage* (CAS). Penelitian yang dilakukan berupaya menerapkan *immutability* dengan tetap mempertimbangkan efisiensi kapasitas penyimpanan.

Berdasarkan penelitian yang dilakukan oleh Múzquiz dkk. (2025), peneliti mengeksplorasi pemanfaatan perangkat penyimpanan *Write-Once-Read-Many* (WORM) untuk menjaga integritas data log. Peneliti menjadikan perangkat keras WORM berbasis optik yang diintegrasikan ke dalam sistem penyimpanan terbuka sebagai objek penelitian. Penelitian menerapkan metode validasi integritas berbasis perangkat keras dengan mekanisme pemeriksaan *checksum* pada tingkat fisik media. Hasil penelitian mengungkapkan bahwa validasi integritas tingkat perangkat keras efektif mencegah modifikasi tanpa izin, sehingga setiap objek tersimpan memiliki jaminan keaslian yang bersifat intrinsik. Penelitian mengangkat

permasalahan terkait kebutuhan perangkat keras khusus yang mahal dan sulit diimplementasikan pada infrastruktur komoditas. Keterbatasan penelitian terdahulu terletak pada ketergantungan terhadap media fisik WORM yang tidak dapat diterapkan secara fleksibel. Penelitian yang dilakukan menawarkan pendekatan berbeda dengan mewujudkan sifat *immutability* secara logis melalui CAS pada perangkat keras komoditas, tanpa ketergantungan pada media fisik khusus.

Udayashankar dkk. (2026) pada penelitiannya menguji akselerasi algoritma *Fast Content-Defined Chunking* (FastCDC) menggunakan instruksi vektor perangkat keras. Peneliti mengoptimalkan implementasi FastCDC menggunakan ekstensi *Single Instruction, Multiple Data* (SIMD) pada prosesor modern sebagai objek penelitian. Penelitian menggunakan metode perbandingan throughput antara implementasi skalar dan implementasi terakselerasi vektor pada dataset berkas berukuran besar. Peneliti menyimpulkan bahwa akselerasi perangkat keras mampu meningkatkan performa pemecahan berkas secara signifikan tanpa mengorbankan rasio deduplikasi. Penelitian mengangkat permasalahan terkait latensi *chunking* yang menjadi hambatan pada sistem cadangan berskala besar. Keterbatasan penelitian terdahulu terletak pada fokus terhadap akselerasi berbasis perangkat keras eksklusif yang tidak selalu tersedia di lingkungan umum. Penelitian yang dilakukan mengadopsi algoritma FastCDC tanpa bergantung pada ekstensi SIMD khusus, sehingga implementasi dapat berjalan pada infrastruktur komoditas standar dengan tetap mempertahankan efisiensi deduplikasi untuk menangani masalah *data bloat*.

Penelitian Lv dkk. (2025) membahas tinjauan komprehensif terhadap arsitektur basis data *key-value* berbasis *Log-Structured Merge-tree* (LSM). Peneliti mengevaluasi berbagai implementasi LSM-tree seperti RocksDB, LevelDB, dan BadgerDB pada beban kerja nyata sistem terdistribusi. Peneliti menggunakan metode survei komparatif terhadap performa tulis, baca, dan efisiensi ruang pada berbagai implementasi LSM. Peneliti menunjukkan bahwa struktur LSM memberikan performa tulis yang stabil dan sangat efisien dalam menangani jutaan referensi *chunk* pada sistem terdistribusi. Penelitian mengangkat permasalahan terkait tingginya beban operasi baca-acak pada sistem metadata berskala besar

akibat proses *compaction*. Keterbatasan penelitian terdahulu terletak pada analisis yang bersifat survei tanpa mengusulkan mekanisme baru yang secara langsung mendukung properti *immutability*. Temuan dari survei menjadi landasan bagi penelitian yang dilakukan dalam memilih BadgerDB sebagai mesin penyimpanan metadata *chunk* dan *manifest* objek pada sistem *Immutable Object Storage*.

Berdasarkan penelitian yang dilakukan oleh Kim dkk. (2026), peneliti menganalisis evolusi perilaku *ransomware* invasif yang secara spesifik menargetkan penghancuran data *cloud repository*. Peneliti menjadikan sampel *ransomware* modern yang memanfaatkan eskalasi hak istimewa untuk menyerang layanan penyimpanan berbasis *cloud* sebagai objek penelitian. Penelitian menerapkan metode analisis perilaku statis dan dinamis terhadap varian *ransomware* pada lingkungan terisolasi. Peneliti menyarankan penggunaan mekanisme validasi berbasis format untuk mendeteksi anomali enkripsi guna memitigasi risiko *credential theft*. Penelitian mengangkat permasalahan terkait ketidakefektifan sistem deteksi berbasis pola terhadap varian *ransomware* yang terus berevolusi. Keterbatasan penelitian terdahulu terletak pada pendekatan detektif yang reaktif, bukan pada aspek pencegahan struktural. Penelitian yang dilakukan mengambil posisi komplementer dengan membangun arsitektur penyimpanan yang secara struktural mencegah kerusakan data, sehingga meskipun deteksi gagal, data cadangan memiliki perlindungan tambahan terhadap manipulasi.

Jin dkk. (2026) pada penelitiannya membahas komparasi performa fungsi kriptografi antara BLAKE3 dan algoritma lama pada arsitektur modern. Peneliti mengevaluasi performa fungsi *hash* SHA-256, SHA-512, MD5, dan BLAKE3 pada beban kerja komputasi intensif. Penelitian menerapkan metode pengukuran *throughput* dan latensi pada berbagai ukuran data masukan. Peneliti menyimpulkan bahwa BLAKE3 menawarkan kecepatan pemrosesan yang jauh lebih tinggi melalui paralelisasi internal berbasis pohon *Merkle*, sehingga sangat cocok untuk proses identifikasi *chunk* pada sistem deduplikasi. Penelitian mengangkat permasalahan terkait tingginya biaya komputasi fungsi *hash* konvensional yang menjadi hambatan performa sistem penyimpanan berskala besar. Keterbatasan penelitian terdahulu terletak pada konteks pengujian yang berfokus pada kecepatan komputasi

secara umum tanpa diterapkan secara konkret pada arsitektur CAS. Penelitian memanfaatkan temuan terdahulu dengan mengadopsi BLAKE3 sebagai fungsi *fingerprinting* pada setiap *chunk* untuk memastikan identitas konten yang unik dan performa *hashing* yang optimal dalam sistem *Immutable Object Storage*.

Penelitian Rahman dkk. (2025) membahas pendekatan arsitektural untuk menjamin autentikasi dan sifat *immutability* pada data transaksi perbankan. Peneliti menjadikan sistem perbankan digital dengan kebutuhan keamanan data transaksi tingkat tinggi sebagai objek penelitian. Penelitian mengusulkan metode rancangan arsitektur multi-lapis yang memisahkan lapisan autentikasi, validasi, dan penyimpanan secara hierarkis. Penelitian menunjukkan bahwa arsitektur multi-lapis efektif mengamankan data dari serangan manipulasi internal. Penelitian mengangkat permasalahan terkait serangan dari dalam (*insider threat*) yang memiliki akses sah namun menyalahgunakan wewenang untuk memanipulasi rekaman transaksi. Keterbatasan penelitian terdahulu terletak pada domain yang terbatas pada sistem perbankan tanpa mempertimbangkan skenario serangan eksternal berbasis *ransomware*. Prinsip pemisahan wewenang pada penelitian terdahulu relevan dengan sistem yang dikembangkan penulis, khususnya dalam merancang mekanisme pemisahan domain kontrol aplikasi dari akses langsung ke penyimpanan fisik.

Berdasarkan penelitian yang dilakukan oleh Higuchi & Kobayashi (2026), peneliti mengkaji dampak *hook point* sistem berkas terhadap rasio pencadangan secara *real-time*. Peneliti mengevaluasi sistem berkas XFS pada *kernel* Linux yang dilengkapi mekanisme pencegahan operasi *file-open hook* untuk pencadangan otomatis. Penelitian menggunakan metode pengukuran rasio berkas tercadangkan pada skenario serangan enkripsi massal. Penelitian membuktikan bahwa menunda penulisan fisik dan mencegah operasi berkas memberikan waktu tambahan bagi sistem untuk memitigasi kerusakan akibat enkripsi *ransomware*. Penelitian mengangkat permasalahan terkait tingginya risiko kehilangan data pada jendela waktu antara modifikasi berkas dan proses pencadangan. Keterbatasan penelitian terdahulu terletak pada pendekatan yang bergantung pada kecepatan respons sistem, bukan pada struktur penyimpanan yang imun terhadap penghapusan.

Penelitian melengkapi pendekatan terdahulu dengan menyediakan repositori tujuan pencadangan yang bersifat *immutable*, sehingga data yang berhasil tercadangkan tidak dapat dihancurkan oleh *ransomware*.

Pipalani dkk. (2025) pada penelitiannya mengeksplorasi ekosistem pengujian repositori berbasis Go. Peneliti mengevaluasi kualitas dan kelengkapan pengujian unit (*unit test*) pada kode sumber Go yang dikembangkan oleh komunitas daring. Penelitian menerapkan metode *fine-tuning* model bahasa besar (LLM) terhadap dataset pengujian Go untuk menghasilkan *unit test* secara otomatis. Penelitian menunjukkan bahwa karakteristik Go mendukung pengembangan pengujian dan pengelolaan kode pada proyek berbasis Go. Temuan tersebut relevan karena sistem yang dikembangkan menggunakan Go untuk menangani operasi baca-tulis secara bersamaan. Penelitian mengangkat permasalahan terkait rendahnya cakupan pengujian otomatis pada proyek Go berskala besar. Keterbatasan penelitian terdahulu terletak pada konteks yang berfokus pada pengujian otomatis, bukan pada performa atau keandalan sistem penyimpanan. Temuan karakteristik Go sebagai bahasa yang unggul dalam konkurensi mendukung keputusan penelitian yang dilakukan untuk mengimplementasikan sistem *Immutable Object Storage* menggunakan Go guna menangani operasi baca-tulis bersamaan secara aman dan efisien.

Penelitian Xu dkk. (2024) membahas pemodelan prediksi berbasis *machine learning* untuk memperkirakan performa baca dan tulis pada berbagai subsistem penyimpanan. Peneliti mengevaluasi subsistem penyimpanan heterogen yang mencakup HDD, SSD, dan NVMe di bawah beban kerja campuran. Penelitian menggunakan metode pemodelan regresi terhadap metrik latensi dan *throughput* berdasarkan karakteristik beban kerja. Peneliti membuktikan bahwa optimalisasi penempatan data berdasarkan pola akses menjamin sistem tetap stabil meskipun menerima beban kerja yang tinggi. Penelitian mengangkat permasalahan terkait sulitnya memprediksi degradasi performa saat subsistem penyimpanan menerima beban kerja campuran secara bersamaan. Keterbatasan penelitian terdahulu terletak pada fokus prediksi performa yang tidak membahas mekanisme perlindungan integritas data. Metodologi pengujian beban yang digunakan oleh Xu dkk. (2023)

menjadi acuan bagi penelitian yang dilakukan dalam merancang skenario pengujian *throughput* dan stabilitas sistem *Immutable Object Storage* di bawah beban kerja tinggi. Tabel 2.1 menyajikan perbandingan sumber pustaka primer.

Tabel 2.1 Sumber Pustaka Primer

No	Judul	Penulis (tahun)	Metode/Alat	Hasil/Kesimpulan
1	<i>VaultFS: Write-once Software Support at the File System Level Against Ransomware Attacks</i>	Pasquale Caporaso, Giuseppe Bianchi, Francesco Quaglia (2024)	VaultFS	Pemisahan kontrol penyimpanan mendukung pencegahan manipulasi cadangan.
2	<i>The Reverse File System: Towards open cost-effective secure WORM storage</i>	Gorka Guardiola Múzquiz, Juan González-Gómez, Enrique Soriano-Salvador (2025)	WORM	Perangkat keras <i>write-once</i> meningkatkan perlindungan data dari penghapusan massal.
3	<i>Accelerating Data Chunking in Deduplication Systems using Vector Instructions</i>	Sreeharsha Udayashankar, Abdelrahman Baba, Samer Al-Kiswany (2026)	FastCDC	Akselerasi vektor meningkatkan kecepatan chunking secara signifikan.
4	<i>Rethinking LSM-tree based Key-Value Stores: A Survey</i>	Yina Lv, Qiao Li, Quanqing Xu, Congming Gao, Chuanhui Yang, Xiaoli Wang, Chun Jason Xue (2025)	LSM-Tree	Struktur LSM sangat efisien untuk mengelola referensi metadata skala besar.
5	<i>Rhea: Detecting Privilege-Escalated Evasive Ransomware Attacks</i>	Beom Heyn Kim, Seok Min Hong (2026)	Analisis Ancaman	Validasi format memitigasi risiko serangan berbasis eskalasi hak istimewa.
6	<i>Trusting What You Cannot See: Auditable Fine-Tuning and Inference</i>	Heng Jin, Shanghao Shi, Chaoyu Zhang, Hexuan Yu, Ning Zhang, Y. Thomas Hou (2026)	BLAKE3	BLAKE3 memberikan performa hashing lebih cepat melalui fitur paralelisasi.

Tabel 2.1 Sumber Pustaka Primer

No	Judul	Penulis (tahun)	Metode/Alat	Hasil/Kesimpulan
7	<i>A Multi-Layer Architecture for Trusted, Verifiable, and Immutable Data</i>	Aufa Nasywa Rahman, Bimo Sunarfri Hantono, Guntur Dharma Putra (2025)	<i>Immutability</i>	Arsitektur mendukung integritas data melalui arsitektur multi-lapis
8	<i>Impact of File-Open Hook Points on Backup Ratio in ROFBS on XFS</i>	Kosuke Higuchi, Ryotaro Kobayashi (2026)	<i>Real-time Backup</i>	Intersepsi operasi berkas memberikan perlindungan tambahan masa pemulihan.
9	<i>Go-UT-Bench: A Fine-Tuning Dataset for LLM-Based Unit Test Generation in Go</i>	Yashshi Pipalani, Rajat Ghosh, Hritik Raj, Vaishnavi Bhargava, Debojyoti Dutta (2025)	Golang	Go memberikan efisiensi tinggi dalam menangani operasi asinkron.
10	<i>ML-based Modeling to Predict I/O Performance on Storage Sub-systems</i>	Yiheng Xu, Pranav Sivaraman, Hariharan Devarajan, Kathryn Mohror, Abhinav Bhatele (2024)	Analisis I/O	Prediksi beban kerja membantu menjaga stabilitas sistem penyimpanan.

Tabel 2.1 menyajikan ringkasan perbandingan penelitian terdahulu. Penulis menyimpulkan bahwa fokus utama penelitian terdahulu belum mencakup integrasi antara mekanisme perlindungan sistem berkas yang tangguh dengan efisiensi kapasitas penyimpanan. Penelitian terdahulu cenderung mengorbankan efisiensi ruang demi mencapai tingkat keamanan tinggi atau sebaliknya. Penelitian menutup celah penelitian terdahulu dengan merancang arsitektur yang menggabungkan deduplikasi berbasis *Content-Addressable Storage* (CAS), algoritma *Fast Content-Defined Chunking* (FastCDC), dan pemisahan domain kontrol antara aplikasi dan penyimpanan fisik. Penelitian yang dilakukan berupaya menerapkan *immutability* secara ketat tanpa memicu pemborosan kapasitas penyimpanan, sehingga memberikan perlindungan optimal terhadap serangan *ransomware*.

2.2 Teori

2.2.1 Immutable Object Storage

Immutable Object Storage adalah sistem penyimpanan objek yang menerapkan sifat permanen pada data yang telah ditulis. Sistem mengidentifikasi setiap objek berdasarkan isi kontennya. Setelah objek tersimpan, data tidak dapat diubah, ditimpa, atau dihapus selama periode retensi yang telah ditentukan. Sistem menjaga integritas data yang telah tersimpan guna mencegah manipulasi oleh peretas yang telah menguasai jalur autentikasi aplikasi. Berbeda dari penyimpanan objek konvensional yang masih mengizinkan penimpaan dengan versi baru, *Immutable Object Storage* dirancang untuk menolak operasi penghapusan atau modifikasi melalui mekanisme arsitektur yang membatasi perubahan data setelah objek tersimpan (Vacca, 2020). Penelitian yang dilakukan mengimplementasikan *Immutable Object Storage* dengan memisahkan proses penulisan melalui layanan penyimpanan yang berkomunikasi menggunakan *Unix Domain Socket* (UDS). Layanan *storage* hanya menerima operasi tulis-tambah, sehingga lapisan aplikasi tidak memiliki jalur langsung untuk menghapus data fisik yang telah tersimpan.

Konsep *immutability* pada penyimpanan objek tidak hanya berkaitan dengan larangan perubahan isi berkas, tetapi juga berkaitan dengan pengendalian siklus hidup objek setelah data masuk ke sistem penyimpanan. Objek yang telah disimpan perlu diperlakukan sebagai unit data permanen, sedangkan perubahan status seperti penghapusan, pemulihan, atau pengarsipan sebaiknya direpresentasikan melalui metadata logis. Pemisahan antara data fisik dan metadata logis membuat sistem tetap dapat menyediakan fungsi pengelolaan berkas kepada pengguna tanpa memberikan hak destruktif langsung terhadap objek yang tersimpan. Pendekatan tersebut relevan dengan prinsip keamanan sistem penyimpanan karena kerusakan pada lapisan aplikasi tidak otomatis memberikan kemampuan untuk mengubah struktur penyimpanan inti.

2.2.2 Write-Once-Read-Many (WORM)

WORM adalah teknologi penyimpanan yang memungkinkan data ditulis hanya satu kali ke perangkat penyimpanan, namun dapat dibaca berulang kali tanpa

batas. Teknologi WORM digunakan untuk arsip kepatuhan regulasi dan perlindungan data kritis karena memberikan jaminan fisik bahwa data tidak akan mengalami modifikasi setelah disimpan. Meskipun efektif, implementasi WORM konvensional menyimpan ulang seluruh berkas setiap kali terjadi perubahan sekecil apa pun, sehingga memicu pemborosan kapasitas yang dikenal sebagai *data bloat* (Silberschatz dkk., 2019). Penelitian yang dilakukan mengambil inspirasi dari prinsip WORM namun menggantikan ketergantungan pada media fisik dengan mekanisme *immutability* logis berbasis CAS, sekaligus mengatasi masalah *data bloat* melalui deduplikasi tingkat *chunk*.

Prinsip WORM memberikan dasar penting bagi sistem penyimpanan yang membutuhkan jaminan integritas jangka panjang. Pada mekanisme WORM, operasi tulis hanya diperbolehkan pada saat data pertama kali disimpan, sedangkan operasi setelahnya dibatasi pada pembacaan data. Karakteristik ini menjadikan WORM sesuai untuk arsip, log audit, dan cadangan yang tidak boleh dimodifikasi setelah proses penyimpanan selesai. Namun, penerapan WORM berbasis objek utuh dapat menimbulkan pemborosan kapasitas ketika data mengalami perubahan kecil, karena sistem tetap memperlakukan versi baru sebagai objek penuh yang berbeda. Oleh karena itu, penelitian menggabungkan prinsip WORM dengan deduplikasi berbasis *chunk* agar perlindungan data tetap berjalan tanpa menyimpan ulang seluruh isi berkas.

2.2.3 Content-Addressable Storage (CAS)

Content-Addressable Storage (CAS) adalah model penyimpanan yang mengidentifikasi dan mengambil setiap unit data berdasarkan nilai *hash* dari isinya, bukan berdasarkan lokasi fisik atau nama berkas. Properti ini membuat dua objek dengan konten identik menghasilkan alamat yang sama, sehingga sistem dapat mendeteksi dan menghindari penulisan duplikat melalui mekanisme deduplikasi. Jika isi objek berubah, nilai *hash*-nya juga berubah, sehingga CAS mendukung sifat *immutability* dan deteksi redundansi data (Buyya dkk., 2011). Penelitian mengimplementasikan CAS sebagai lapisan inti sistem. Lapisan inti mengalamatkan setiap *chunk* yang dihasilkan FastCDC berdasarkan nilai *hash* BLAKE3 sebelum disimpan ke direktori penyimpanan fisik.

CAS memiliki hubungan langsung dengan prinsip integritas data karena alamat penyimpanan diturunkan dari isi data. Apabila isi data berubah, nilai *hash* yang menjadi alamat data juga berubah, sehingga manipulasi terhadap data dapat terdeteksi melalui ketidaksesuaian antara alamat dan konten. Model ini berbeda dari penyimpanan berbasis nama berkas atau lokasi direktori karena identitas data tidak bergantung pada *path*, melainkan pada konten aktual yang tersimpan. Karakteristik CAS memungkinkan sistem deduplikasi memeriksa keberadaan data sebelum penulisan *chunk* baru dilakukan. Dengan demikian, CAS mendukung dua kebutuhan utama penelitian, yaitu menjaga keutuhan objek dan mengurangi duplikasi penyimpanan.

2.2.4 Content-Defined Chunking

Deduplikasi data adalah proses menghilangkan salinan redundan dari data yang sama untuk mengoptimalkan kapasitas penyimpanan. Teknik deduplikasi dapat dilakukan pada tingkat berkas, blok dengan ukuran tetap, atau potongan berukuran variabel berdasarkan konten. *Content-Defined Chunking* (CDC) adalah teknik pemecahan berkas menjadi potongan-potongan (*chunk*) berukuran variabel berdasarkan pola konten, dengan menggeser jendela *hash* untuk mendeteksi batas pemisahan alami. Pendekatan CDC umumnya menghasilkan rasio deduplikasi yang lebih tinggi dibandingkan pemecahan berbasis ukuran tetap, karena pergeseran konten tidak mengubah identitas *chunk* yang tidak berubah (Xia dkk., 2016). Dalam penelitian yang dilakukan, sistem menggunakan CDC sebagai mesin pemecah berkas sebelum setiap *chunk* diproses lebih lanjut oleh mesin CAS untuk deduplikasi dan penyimpanan permanen.

Keunggulan CDC terletak pada kemampuannya menjaga kestabilan batas *chunk* ketika terjadi penyisipan, penghapusan, atau perubahan kecil pada bagian tertentu dari berkas. Pada metode *fixed-size chunking*, penambahan beberapa *byte* di awal berkas dapat menggeser seluruh batas blok setelah titik perubahan, sehingga banyak blok lama tidak lagi cocok dengan versi sebelumnya. CDC mengurangi masalah tersebut karena batas *chunk* ditentukan berdasarkan pola konten, bukan semata-mata berdasarkan ukuran tetap. Akibatnya, bagian berkas yang tidak berubah masih berpeluang menghasilkan *chunk* yang sama dan dapat digunakan

kembali oleh sistem. Karakteristik ini sangat penting dalam skenario *backup*, karena perubahan data umumnya bersifat parsial dan berulang dari waktu ke waktu.

2.2.5 Fast Content-Defined Chunking (FastCDC)

FastCDC adalah algoritma CDC yang dirancang untuk meningkatkan kecepatan pemecahan berkas dengan tetap mempertahankan rasio deduplikasi. FastCDC mengoptimalkan performa dengan menggunakan jendela geser berbobot dan fungsi *hash* sederhana untuk menentukan batas *chunk*, serta menerapkan kebijakan ukuran minimum dan maksimum *chunk* untuk mengendalikan distribusi ukuran. Pendekatan memungkinkan deduplikasi tingkat *chunk* yang sangat efisien dalam menghemat kapasitas penyimpanan, khususnya pada berkas yang mengalami perubahan parsial (Xia dkk., 2016). Penelitian yang dilakukan menggunakan FastCDC sebagai tahap pertama alur penyimpanan objek. Setiap berkas yang masuk dipecah menjadi *chunk* oleh FastCDC sebelum masing-masing *chunk* diproses oleh mesin CAS untuk pengecekan duplikasi dan penyimpanan permanen.

FastCDC dikembangkan untuk mengurangi biaya komputasi yang sering muncul pada algoritma CDC tradisional. Algoritma ini mempercepat proses pencarian batas *chunk* dengan menyederhanakan operasi *hash* dan mengatur distribusi ukuran *chunk* melalui batas minimum, rata-rata, dan maksimum. Pengaturan ukuran tersebut membantu sistem menghindari *chunk* yang terlalu kecil maupun terlalu besar. *Chunk* yang terlalu kecil dapat meningkatkan jumlah metadata dan beban indeks, sedangkan *chunk* yang terlalu besar dapat menurunkan rasio deduplikasi. Oleh karena itu, FastCDC sesuai digunakan pada sistem penyimpanan cadangan karena mampu menjaga keseimbangan antara kecepatan pemrosesan, ukuran metadata, dan efisiensi deduplikasi.

2.2.6 Kriptografi dan Fungsi Hash

Kriptografi adalah ilmu yang mempelajari teknik pengamanan informasi melalui transformasi matematis. Salah satu primitif kriptografi yang fundamental adalah fungsi *hash*, yaitu fungsi satu arah yang memetakan data masukan berukuran sembarang menjadi keluaran berukuran tetap yang disebut *digest*. Fungsi *hash* kriptografi memiliki tiga sifat utama yaitu resistansi terhadap pra-citra atau

preimage resistance, resistansi terhadap pra-citra kedua atau *second preimage resistance*, dan resistansi terhadap kolisi atau *collision resistance*. Ketiga sifat utama fungsi hash membuat perubahan kecil pada data masukan sangat mungkin menghasilkan *digest* yang berbeda secara signifikan. (Stallings dkk., 2023). Dalam penelitian yang dilakukan, penulis menggunakan fungsi *hash* sebagai mekanisme penentuan identitas setiap *chunk* pada sistem CAS, sekaligus sebagai instrumen verifikasi integritas data.

Pada sistem CAS, fungsi *hash* tidak hanya berperan sebagai alat kriptografi, tetapi juga menjadi mekanisme pengalamatan data. Nilai *digest* digunakan sebagai identitas unik yang menghubungkan *chunk*, *manifest*, dan proses rekonstruksi objek. Apabila terdapat perubahan satu bit pada *chunk*, *digest* yang dihasilkan akan berbeda, sehingga sistem dapat mendeteksi ketidaksesuaian antara metadata *manifest* dan isi *chunk* fisik. Sifat *collision resistance* menjadi penting karena dua *chunk* berbeda seharusnya tidak menghasilkan identitas yang sama. Dengan memanfaatkan fungsi *hash*, penelitian dapat menghubungkan aspek keamanan data dengan efisiensi penyimpanan melalui satu mekanisme identifikasi konten.

2.2.7 BLAKE3

BLAKE3 adalah fungsi *hash* kriptografi yang dirancang untuk kecepatan tinggi dan keamanan yang kuat. BLAKE3 menggunakan struktur pohon *Merkle* yang memungkinkan pemrosesan data secara paralel pada berbagai ukuran blok, menjadikannya jauh lebih cepat daripada SHA-256 maupun SHA-512 pada perangkat keras modern. BLAKE3 juga bersifat dapat diperluas (*extendable output*) sehingga panjang keluaran dapat disesuaikan dengan kebutuhan (Jack O'Connor dkk., 2021). Dalam penelitian yang dilakukan, penulis memilih BLAKE3 sebagai fungsi *fingerprinting* pada sistem CAS untuk menghasilkan identitas unik berbasis *hash* setiap *chunk*, memastikan performa *hashing* yang optimal pada operasi tulis dengan volume tinggi.

Pemilihan BLAKE3 relevan dengan kebutuhan sistem penyimpanan karena proses *hashing* dilakukan berulang terhadap setiap *chunk* yang dihasilkan oleh FastCDC. Semakin banyak *chunk* yang diproses, semakin besar pengaruh kecepatan *hash* terhadap keseluruhan waktu unggah dan penyimpanan. Struktur

Merkle-tree pada BLAKE3 memungkinkan pemrosesan data secara paralel, sehingga cocok untuk lingkungan yang membutuhkan *throughput* tinggi. Selain itu, *digest* yang dihasilkan dapat digunakan sebagai *fingerprint* konten dalam proses deduplikasi. Dengan demikian, BLAKE3 mendukung fungsi ganda dalam penelitian, yaitu mempercepat identifikasi *chunk* dan membantu menjaga integritas data pada sistem CAS.

2.2.8 Basis Data

Basis data adalah kumpulan data terorganisasi yang disimpan secara sistematis dan dapat diakses, dikelola, serta diperbarui secara efisien. Model basis data diklasifikasikan berdasarkan cara pengorganisasian datanya, meliputi model relasional, dokumen, *key-value*, grafik, dan kolom lebar. Basis data *key-value* menyimpan data sebagai pasangan kunci-nilai dan menawarkan latensi baca-tulis yang sangat rendah, sehingga cocok untuk aplikasi yang memerlukan akses metadata berskala tinggi (Kleppmann & Riccomini Chris, 2026). Dalam penelitian yang dilakukan, sistem menggunakan basis data untuk menyimpan metadata *chunk* dan *manifest* objek yang diperlukan untuk operasi rekonstruksi berkas.

Perancangan basis data pada sistem penyimpanan tidak hanya berhubungan dengan penyimpanan data pengguna, tetapi juga dengan konsistensi metadata yang menentukan keberhasilan rekonstruksi objek. Metadata yang tidak konsisten dapat menyebabkan objek tidak dapat dipulihkan meskipun *chunk* fisik masih tersedia. Oleh karena itu, sistem perlu membedakan basis data yang menyimpan metadata aplikasi dan basis data yang menyimpan metadata penyimpanan inti. Pemisahan tersebut membantu membatasi dampak kerusakan ketika lapisan aplikasi mengalami gangguan. Penelitian menggunakan PostgreSQL untuk data aplikasi yang dinamis, sedangkan BadgerDB digunakan untuk indeks *chunk* dan *manifest* yang berhubungan langsung dengan *Vault Core*.

2.2.9 BadgerDB

BadgerDB adalah basis data *key-value* sumber terbuka yang diimplementasikan dalam Go dan dioptimalkan untuk performa tulis tinggi menggunakan struktur *LSM-Tree*. BadgerDB memisahkan penyimpanan kunci

(*key*) dan nilai (*value*) ke berkas yang berbeda, strategi yang dikenal sebagai *key-value separation*, untuk mengurangi *write amplification* pada proses *compaction*. BadgerDB mendukung transaksi ACID dan operasi iterasi yang efisien (Kleppmann & Riccomini Chris, 2026). Dalam penelitian yang dilakukan, sistem menggunakan BadgerDB untuk mengelola dua jenis metadata yaitu indeks keberadaan *chunk* berdasarkan nilai BLAKE3-nya dan *manifest* objek yang mencatat daftar *chunk* menyusun setiap berkas yang tersimpan.

BadgerDB sesuai untuk penyimpanan metadata *chunk* karena pola akses pada sistem CAS banyak melibatkan operasi pencarian berdasarkan kunci. Nilai *hash* BLAKE3 dapat digunakan sebagai *key*, sedangkan informasi lokasi *chunk*, ukuran *chunk*, dan referensi *manifest* dapat disimpan sebagai *value*. Struktur *key-value* membantu sistem melakukan pengecekan duplikasi secara cepat sebelum menulis data baru ke penyimpanan fisik. Selain itu, dukungan transaksi membantu menjaga konsistensi saat sistem membentuk *manifest* objek setelah seluruh *chunk* selesai diproses. Mekanisme ini penting untuk mencegah kondisi ketika sebagian *chunk* telah tersimpan, tetapi objek belum sah dianggap valid karena *manifest* belum terbentuk secara lengkap.

2.2.10 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional tingkat lanjut yang menggunakan dan memperluas standar bahasa pemrograman SQL. Sistem mengelola relasi entitas terstruktur secara aman dan memastikan integritas data melalui implementasi prinsip ACID (*Atomicity, Consistency, Isolation, Durability*) secara ketat pada setiap operasi penyisipan atau pembaruan (Drake & Worsley, 2011). Dalam penelitian yang dilakukan, penulis menggunakan PostgreSQL pada lingkungan publik atau *Environment A* untuk mengelola informasi dinamis seperti kredensial pengguna dan hierarki direktori secara terisolasi dari repositori utama *Vault*.

2.2.11 Go (Golang)

Go adalah bahasa pemrograman sumber terbuka yang dirancang oleh Google dengan fokus pada efisiensi eksekusi, kesederhanaan sintaksis, dan

kemudahan dalam pemrograman konkurensi. Go menyediakan fitur *goroutine* sebagai unit konkurensi ringan yang dikelola oleh *runtime* Go, serta mekanisme *channel* untuk komunikasi antar *goroutine* secara aman. Go juga menyediakan *garbage collection*, manajemen memori yang aman, serta pustaka standar yang kaya untuk pengembangan layanan jaringan dan sistem penyimpanan berperforma tinggi (Bodner, 2021). Penelitian yang dilakukan mengimplementasikan seluruh komponen sistem dalam Go, memanfaatkan *goroutine* untuk memproses operasi FastCDC dan penulisan *chunk* secara bersamaan, serta *Unix Domain Socket* (UDS) untuk komunikasi antara lapisan API dan mesin penyimpanan.

2.2.12 React

React adalah pustaka JavaScript sumber terbuka yang dirancang untuk membangun antarmuka pengguna secara deklaratif dan berbasis komponen. React mengelola status aplikasi secara efisien dengan memperbarui komponen visual secara spesifik saat terjadi perubahan data tanpa memicu proses pemuatan ulang pada keseluruhan halaman peramban (Banks & Porcello, 2020). Dalam penelitian yang dilakukan, penulis mengimplementasikan pustaka React untuk membangun antarmuka halaman dasbor dan pratinjau berkas guna menyajikan status penguncian objek serta metrik deduplikasi secara langsung pada antarmuka pengguna.

2.2.13 Ransomware

Ransomware adalah jenis perangkat lunak berbahaya (*malware*) yang mengenkripsi data korban menggunakan kunci yang hanya diketahui oleh penyerang, kemudian meminta pembayaran tebusan sebagai syarat pemulihan. Varian *ransomware* modern menggunakan strategi pemusnahan cadangan secara sistematis yaitu penyerang terlebih dahulu mengidentifikasi dan menghancurkan repositori *backup* sebelum mengenkripsi data utama, sehingga korban tidak memiliki pilihan pemulihan tanpa membayar tebusan. Risiko serangan ini meningkat ketika arsitektur cadangan masih menggunakan jalur autentikasi yang sama antara aplikasi dan penyimpanan fisik, sehingga kredensial yang diretas memberikan akses penuh ke repositori cadangan (Stallings dkk., 2023). Penelitian

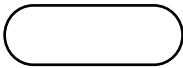
yang dilakukan memitigasi vektor serangan ini dengan menerapkan arsitektur yang dirancang untuk menolak operasi penghapusan dari jalur aplikasi.

Ancaman *ransomware* terhadap repositori cadangan tidak hanya muncul dari proses enkripsi data utama, tetapi juga dari penyalahgunaan hak akses yang sah setelah kredensial pengguna atau server aplikasi berhasil dikuasai. Pada kondisi tersebut, penyerang tidak selalu perlu mengeksploitasi kerentanan teknis pada media penyimpanan, karena operasi penghapusan atau *penimpaan* dapat dilakukan melalui fitur aplikasi yang memang tersedia. Oleh karena itu, mitigasi *ransomware* pada sistem cadangan perlu dirancang pada tingkat arsitektur, bukan hanya pada tingkat autentikasi. Penelitian menempatkan *Vault Core* sebagai komponen yang memiliki aturan operasi lebih ketat dibandingkan lapisan aplikasi, sehingga penguasaan API Service tidak otomatis memberikan kemampuan untuk menghapus *chunk*, *manifest*, atau objek fisik.

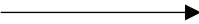
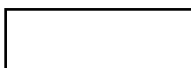
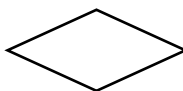
2.2.14 Flowchart

Flowchart adalah penggambaran grafis dari langkah-langkah dan urutan prosedur dalam suatu program. *Flowchart* membantu analis dan programmer untuk memecahkan masalah menjadi segmen-segmen yang lebih kecil dan menganalisis alternatif dalam operasional. Dengan menggunakan simbol-simbol tertentu, *flowchart* menggambarkan urutan proses secara detail dan hubungan antara instruksi dalam suatu program. *Flowchart* juga mempermudah penyelesaian masalah, terutama yang memerlukan studi dan evaluasi lebih lanjut. Perangkat lunak pembelajaran *flowchart* dirancang untuk membantu pengguna memahami algoritma dan proses pembuatan *flowchart* tanpa perlu menulis kode program, serta menyediakan objek-objek yang diperlukan untuk membuat diagram alir. Berikut dilampirkan simbol-simbol *flowchart* yang tersedia pada Tabel 2.2 (Zalukhu dkk., 2023).

Tabel 2.2 Simbol Flowchart

Simbol	Nama	Fungsi
	Terminator	Permulaan/akhir program

Tabel 2.2 Simbol Flowchart

Simbol	Nama	Fungsi
	Garis alir (<i>flow line</i>)	Arah aliran program
	<i>Preparation</i>	Proses inisialisasi/pemberian harga awal
	Proses	Proses perhitungan/proses pengolahan data
	Input/output data	Proses <i>input/output</i> data, parameter, informasi
	Sub program (<i>predefined process</i>)	Permulaan sub program/proses menjalankan sub program
	<i>Decision</i>	Perbandingan pernyataan, penyeleksian data yang memberikan pilihan untuk langkah selanjutnya
	<i>On page connector</i>	Penghubung bagian-bagian <i>flowchart</i> yang berada pada satu halaman
	<i>Off page connector</i>	Penghubung bagian-bagian <i>flowchart</i> yang berada pada halaman berbeda

Sumber: Zalukhu (2023)

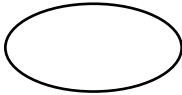
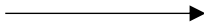
Pada Tabel 2.2 menyajikan simbol-simbol standar yang digunakan dalam *flowchart* untuk mendeskripsikan alur dan proses sistem secara sistematis. Simbol "*Terminator*" digunakan untuk merepresentasikan awal atau akhir dari suatu proses, sedangkan "Garis Alir (*Flow Line*)" menunjukkan arah aliran proses dalam diagram. "*Preparation*" menggambarkan proses inisialisasi atau pemberian nilai awal, sementara "Proses" digunakan untuk mengilustrasikan tahapan pengolahan data. Simbol "*Input/Output Data*" menandakan aktivitas masukan atau keluaran

data, dan "*Predefined Process*" merepresentasikan sub-program atau proses yang telah didefinisikan sebelumnya. "*Decision*" digunakan untuk proses pengambilan keputusan berdasarkan kondisi tertentu, dan "*On Page Connector*" serta "*Off Page Connector*" bertindak sebagai penghubung antar bagian *flowchart*, baik di halaman yang sama maupun halaman yang berbeda.

2.2.15 Data Flow Diagram

Data Flow Diagram (DFD) adalah alat yang digunakan untuk menggambarkan aliran data dalam suatu sistem yang berinteraksi dengan lingkungannya, membantu dalam mengidentifikasi kebutuhan pengguna dan merancang sistem dengan fokus pada struktur dan proses kerja. DFD sering digunakan dalam pengembangan perangkat lunak berbasis metodologi seperti *System Development Life Cycle* (SDLC) dan *Structured System Analysis and Design Methodology* (SSADM). DFD memiliki komponen penting seperti entitas, proses, arus data, dan data store, yang berfungsi untuk memvisualisasikan bagaimana data bergerak melalui sistem yang disajikan dengan Tabel 2.3 (Ridwan dkk., 2022).

Tabel 2.3 Simbol Data Flow Diagram

Simbol	Nama
	External Entity
	Process
	Data Storage
	Data Flow

Sumber: Ridwan (2022)

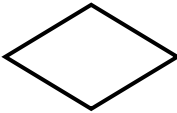
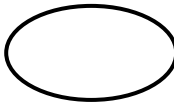
Tabel 2.3 menggambarkan simbol-simbol standar yang digunakan dalam *Data Flow Diagram* (DFD) untuk merepresentasikan elemen-elemen utama dalam analisis sistem. Simbol "*External Entity*" berbentuk oval digunakan untuk

merepresentasikan entitas eksternal yang berinteraksi dengan sistem, seperti pengguna atau organisasi lain. Simbol "*Process*" berbentuk persegi panjang menunjukkan aktivitas atau fungsi dalam sistem yang mengolah data untuk menghasilkan *output*. "*Data Storage*" direpresentasikan dengan simbol dua garis sejajar, yang menunjukkan tempat penyimpanan data dalam sistem, baik secara sementara maupun permanen. Terakhir, simbol "*Data Flow*" berupa panah menunjukkan arah aliran data antara entitas, proses, dan penyimpanan.

2.2.16 Entity Relationship Diagram

Entity Relationship Diagram (ERD) adalah teknik pemodelan basis data untuk menggambarkan hubungan antar entitas dalam suatu sistem. ERD berfungsi sebagai representasi grafis dari model data konseptual yang mencerminkan kebutuhan data pengguna dalam sistem basis data. Dalam ERD, terdapat tiga elemen dasar: entitas, atribut, dan relasi. Entitas adalah objek yang dapat berupa manusia, tempat, atau benda, yang diwakili oleh simbol persegi panjang. ERD merupakan tahap awal dalam desain basis data dan penting untuk memastikan bahwa semua entitas saling terhubung dengan benar, serta untuk menghindari kesalahan konseptual, prosedural, dan teknis dalam pembuatan basis data. Berikut dilampirkan simbol-simbol ERD yang tersedia pada Tabel 2.4 (Pulungan dkk., 2022).

Tabel 2.4 Simbol Entity Relationship Diagram

Nama	Gambar	Keterangan
Entitas		Entitas merupakan objek yang dapat didefinisikan dalam lingkungan sistem
Relasi		Relasi, menunjukkan adanya hubungan diantara sejumlah entitas yang berbeda.
Atribut		Atribut, mendeskripsikan karakter entitas (atribut yang berfungsi sebagai kunci yang diberi garis bawah).

Nama	Gambar	Keterangan
Garis		Garis, penghubung antara relasi dengan entitas, relasi dan entitas dengan atribut.

Sumber: Pulungan (2022)

Tabel 2.4 menjelaskan simbol-simbol dasar dalam *Entity Relationship Diagram* (ERD) yang digunakan untuk memodelkan struktur dan hubungan data dalam sistem basis data. Simbol "Entitas" yang direpresentasikan dengan persegi panjang, merepresentasikan objek yang dapat didefinisikan, seperti "Pengguna" atau "Produk". Simbol "Relasi" berbentuk wajik, menggambarkan hubungan antara dua atau lebih entitas, misalnya relasi "Memesan" antara entitas "Pelanggan" dan "Produk". Atribut entitas atau relasi direpresentasikan dengan oval, yang mendeskripsikan karakteristik atau properti entitas. Atribut kunci diberi garis bawah untuk menandai identitas unik suatu entitas. Garis digunakan sebagai penghubung antara entitas, relasi, dan atribut.

2.2.17 Pengujian Black Box

Pengujian *black box* adalah metode pengujian perangkat lunak yang mengevaluasi fungsionalitas sistem berdasarkan spesifikasi keluaran yang diharapkan, tanpa memperhatikan struktur atau logika internal kode program. Penguji berinteraksi dengan sistem semata-mata melalui antarmuka yang tersedia, kemudian membandingkan keluaran aktual dengan keluaran yang diharapkan sesuai spesifikasi. Pengujian *black box* efektif mendeteksi ketidaksesuaian fungsi, kesalahan antarmuka, dan perilaku yang menyimpang dari spesifikasi tanpa bergantung pada pengetahuan implementasi (Pressman & Maxim, 2020). Penelitian menerapkan pengujian *black box* pada fungsi utama sistem *Immutable Object Storage* untuk memverifikasi kesesuaian keluaran terhadap skenario unggah, unduh, deduplikasi, *soft delete*, pemulihan objek, serta penanganan masukan tidak valid dan kegagalan proses unggah.

2.2.18 Pengujian Keamanan

Pengujian keamanan (*security testing*) adalah proses evaluasi sistematis yang bertujuan memverifikasi bahwa mekanisme pertahanan suatu sistem bekerja

sesuai klaim dan mengidentifikasi kerentanan sebelum sistem dieksploitasi secara nyata. Pengujian keamanan mencakup simulasi serangan terhadap lapisan autentikasi, otorisasi, isolasi hak akses, dan ketahanan terhadap manipulasi data, dengan mengacu pada kerangka metodologis seperti *OWASP Testing Guide* dan *NIST SP 800-115* (Stallings dkk., 2023). Dalam penelitian yang dilakukan, penulis merancang pengujian keamanan untuk memvalidasi mekanisme pertahanan sistem terhadap skenario serangan *ransomware*. Pengujian dilakukan dengan mensimulasikan kondisi pasca-peretasan, yaitu saat penyerang menguasai lapisan API dan mencoba menjalankan operasi penghapusan massal atau penimpaan objek melalui jalur *Unix Domain Socket* (UDS). Pengujian ini bertujuan untuk mengevaluasi apakah pemisahan domain kontrol mampu menolak operasi destruktif yang tidak sesuai dengan rancangan sistem.

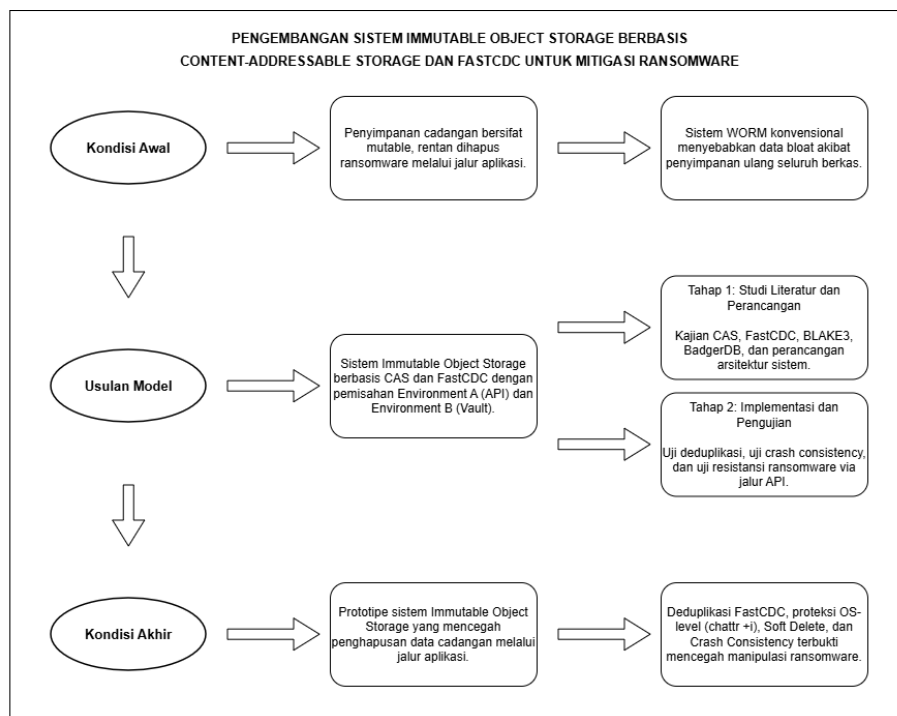
Pengujian keamanan pada penelitian tidak diarahkan untuk membuktikan bahwa sistem bebas dari seluruh kerentanan, melainkan untuk memvalidasi klaim utama rancangan, yaitu pembatasan operasi destruktif terhadap repositori penyimpanan. Skenario pengujian perlu menempatkan penyerang seolah-olah telah memiliki akses ke lapisan aplikasi, kemudian menguji apakah akses tersebut dapat digunakan untuk menghapus, menimpa, atau memanipulasi data fisik pada *Vault Core*. Dengan pendekatan tersebut, pengujian menjadi lebih sesuai dengan konteks *ransomware* modern yang sering menargetkan *backup* setelah memperoleh akses ke lingkungan aplikasi.

BAB III

METODE PENELITIAN

3.1 Kerangka Penelitian

Penulis menyusun kerangka penelitian untuk merumuskan tahapan pengembangan sistem secara terstruktur. Penulis mengawali kerangka penelitian dengan menetapkan kondisi awal melalui proses identifikasi kerentanan repositori cadangan terhadap serangan *ransomware*. Setelah merumuskan akar permasalahan, penulis mengumpulkan data teknis dan merancang arsitektur sistem penyimpanan hibrida sebagai solusi perlindungan data. Tahap penyelesaian penelitian berfokus pada pencapaian kondisi ideal melalui pengujian implementasi untuk mengevaluasi keamanan dan fungsionalitas prototipe. Gambar 3.1 menunjukkan kerangka penelitian.



Gambar 3.1 Kerangka Penelitian

Gambar 3.1 menunjukkan alur kerangka penelitian yang digunakan dalam pengembangan sistem *Immutable Object Storage*. Penjabaran tahapan kerangka penelitian diuraikan secara berurutan dari awal hingga akhir. Tahap kondisi awal mencakup langkah penulis dalam mengidentifikasi masalah dengan menganalisis

kerentanan arsitektur penyimpanan cadangan terhadap serangan *ransomware*. Langkah selanjutnya adalah tahap pengumpulan data primer dan sekunder untuk menentukan spesifikasi kebutuhan sistem. Berdasarkan data yang terkumpul, penulis melakukan perancangan arsitektur *Immutable Object Storage* yang mengintegrasikan mekanisme *Content-Addressable Storage* (CAS) dan algoritma FastCDC. Penulis kemudian mengimplementasikan rancangan sistem ke dalam bentuk prototipe perangkat lunak. Setelah tahap implementasi selesai, penelitian menguji sistem menggunakan pengujian *black box* dan pengujian keamanan. Tahap kondisi akhir diarahkan untuk menghasilkan prototipe sistem penyimpanan data yang dapat diuji dalam memitigasi risiko modifikasi paksa oleh *ransomware* secara struktural. Evaluasi penelitian difokuskan pada mekanisme penyimpanan, deduplikasi, rekonstruksi objek, dan ketahanan terhadap manipulasi data, sedangkan antarmuka web hanya digunakan untuk menjalankan skenario demonstrasi. Pengujian sistem dilakukan melalui beberapa skenario yang merepresentasikan kondisi normal dan kondisi tidak normal. Kondisi normal meliputi unggah berkas baru, unggah berkas identik, unggah berkas dengan perubahan sebagian, dan pengunduhan kembali objek. Skenario tersebut digunakan untuk mengevaluasi proses *Fast Content-Defined Chunking* (FastCDC), perhitungan hash BLAKE3, penyimpanan berbasis *Content-Addressable Storage* (CAS), deduplikasi, pembentukan *manifest* objek, serta rekonstruksi data. Kondisi normal meliputi unggah berkas baru, unggah berkas identik, unggah berkas dengan perubahan sebagian, pengunduhan kembali objek, *soft delete*, dan pemulihan item. Kondisi tidak normal meliputi permintaan *manifest* tidak valid, proses unggah yang dihentikan sebelum tahap *commit*, serta simulasi manipulasi melalui lapisan API. Pengujian kondisi tidak normal digunakan untuk menilai konsistensi metadata, ketahanan terhadap manipulasi *ransomware*, serta memastikan bahwa *soft delete* hanya mengubah metadata logis tanpa menghapus *chunk* fisik atau *manifest* pada *Vault Core*.

3.2 Data Penelitian

3.2.1 Sumber Data

Penulis menggunakan dua klasifikasi sumber data, yakni data primer dan data sekunder. Penulis mengumpulkan data primer secara langsung dari objek penelitian, sedangkan data sekunder diperoleh dari referensi teknis, dokumentasi resmi, buku, dan artikel ilmiah yang relevan. Sebagai data primer, penulis menggunakan kumpulan sampel berkas biner, dokumen teks, dan gambar untuk menguji performa pemecahan data (*chunking*). Tabel 3.1 menyajikan rincian sumber data penelitian.

Tabel 3.1 Sumber Data Penelitian

Jenis Data	Kategori	Deskripsi	Sumber
Primer	Sampel Uji Coba	Data primer berupa sampel berkas simulasi yang digunakan untuk menguji mekanisme penyimpanan <i>Immutable Object Storage</i> . Sampel terdiri dari berkas identik, berkas dengan perubahan sebagian, dan berkas berbeda sepenuhnya. Kategori tersebut digunakan untuk mengevaluasi kemampuan FastCDC dalam menghasilkan <i>chunk</i> , kemampuan CAS dalam mendeteksi duplikasi, rasio deduplikasi, keberhasilan rekonstruksi objek, serta konsistensi metadata setelah kegagalan proses unggah.	Simulasi mandiri oleh penulis pada lingkungan komputasi lokal.
Sekunder	Literatur dan dokumentasi teknis	Jurnal, buku, dokumentasi teknis, serta laporan keamanan yang digunakan sebagai dasar teori CAS, FastCDC, WORM, BLAKE3, dan mitigasi <i>ransomware</i> .	Studi pustaka dari artikel ilmiah, dokumentasi resmi, dan sumber teknis terkait.

Sampel data primer dirancang untuk merepresentasikan skenario penyimpanan cadangan yang memiliki tingkat kemiripan konten berbeda. Penulis menggunakan berkas identik untuk menguji kemampuan deduplikasi penuh, berkas dengan perubahan sebagian untuk menguji kemampuan FastCDC dalam mempertahankan *chunk* yang tidak berubah, serta berkas berbeda sepenuhnya untuk menguji proses penulisan *chunk* baru. Selain itu, penulis menyiapkan skenario unggah tidak selesai untuk mengevaluasi konsistensi metadata dan *manifest* saat terjadi kegagalan proses penyimpanan.

3.2.2 Mendapatkan Data

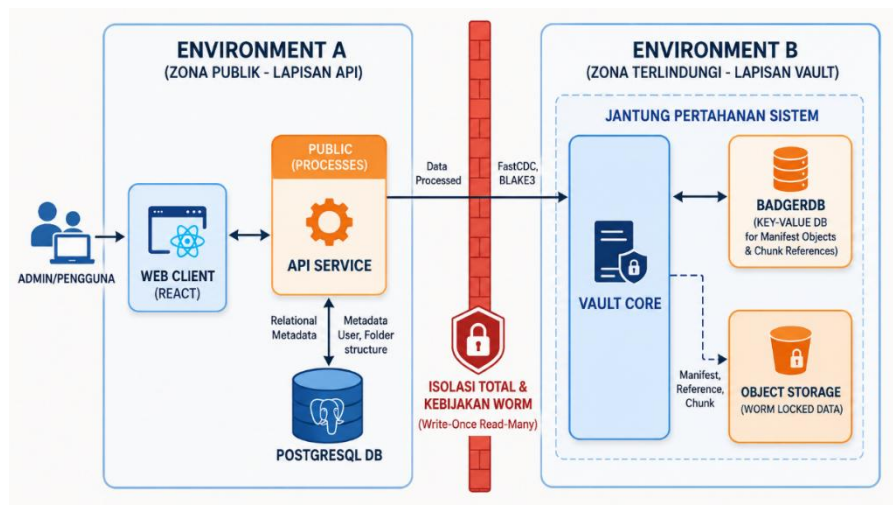
Penulis memperoleh data primer melalui pembuatan sampel berkas simulasi secara mandiri pada lingkungan komputasi lokal. Sampel disusun ke dalam beberapa skenario, yaitu berkas identik, berkas dengan perubahan sebagian, berkas berbeda sepenuhnya, dan berkas yang proses unggahnya dihentikan sebelum selesai. Berkas identik digunakan untuk menguji deduplikasi penuh, berkas dengan perubahan sebagian digunakan untuk menguji penggunaan ulang *chunk* yang tidak berubah, sedangkan berkas berbeda sepenuhnya digunakan untuk menguji pembentukan *chunk* baru dan *manifest* objek baru. Skenario unggah tidak selesai digunakan untuk menguji apakah sistem tetap menjaga konsistensi metadata dan tidak menandai objek sebagai valid sebelum proses pembentukan *manifest* selesai.

3.2.3 Waktu Pengumpulan Data

Penulis melaksanakan proses pengumpulan data selama empat minggu, terhitung sejak minggu pertama hingga minggu keempat masa pelaksanaan Proyek Utama Informatika. Penulis mengalokasikan jadwal pengumpulan data secara paralel dengan tahapan analisis masalah dan perumusan studi literatur. Strategi penjadwalan paralel bertujuan memastikan penulis menguasai seluruh spesifikasi teknis dan algoritma pendukung sebelum memulai tahap implementasi kode program.

3.3 Arsitektur Model

Arsitektur model menggambarkan struktur hibrida yang memisahkan lingkungan publik dari lingkungan penyimpanan. Penulis merancang sistem dengan membagi domain kontrol ke dalam dua zona isolasi guna memitigasi risiko modifikasi data oleh *ransomware*. Gambar 3.2 menunjukkan arsitektur model *Immutable Object Storage*.



Gambar 3.2 Arsitektur Model Sistem

Environment A beroperasi sebagai zona publik yang menangani seluruh interaksi pengguna. Lingkungan publik terdiri dari tiga komponen utama, yaitu *Web Client*, *API Service*, dan basis data relasional PostgreSQL. Web Client yang dibangun menggunakan *React* berfungsi sebagai antarmuka demonstrasi untuk menjalankan proses unggah, melihat metadata dasar, menampilkan status objek, dan melakukan pengunduhan kembali berkas. Antarmuka ini tidak menjadi fokus evaluasi penelitian karena kontribusi utama penelitian berada pada *Vault Core* sebagai sistem penyimpanan *immutable*. *API Service* menerima permintaan dari klien, mengeksekusi logika autentikasi, serta mengelola metadata relasional pada basis data PostgreSQL. Basis data PostgreSQL menyimpan informasi dinamis seperti identitas pengguna, struktur folder, dan pemetaan kepemilikan berkas.

Environment B beroperasi sebagai *Vault Core* yang terisolasi untuk mengelola penyimpanan fisik dan metadata permanen. Saat proses unggah

berlangsung, API Service menerima aliran data dari pengguna dan meneruskannya ke *Vault Core* melalui jalur komunikasi lokal yang dibatasi. *Vault Core* kemudian menjalankan algoritma FastCDC untuk memecah berkas menjadi *chunk*, menghitung nilai *hash* BLAKE3, memeriksa deduplikasi, serta menyimpan *chunk* baru ke direktori penyimpanan fisik. *Vault Core* menggunakan BadgerDB sebagai basis data *key-value* untuk mencatat *manifest* objek dan referensi *chunk*. Integrasi mekanisme tulis-tambah, pemisahan hak akses, dan isolasi domain kontrol dirancang untuk menjaga integritas data meskipun lapisan aplikasi pada Environment A mengalami peretasan.

3.4 Analisis dan Perancangan

3.4.1 Kebutuhan Fungsional

Kebutuhan fungsional menjelaskan interaksi antara pengguna dan sistem, yang dikelompokkan menjadi tiga bagian utama, yakni kebutuhan masukan, kebutuhan proses, dan kebutuhan luaran.

a. Kebutuhan Masukan

Kebutuhan masukan mendefinisikan seluruh data atau instruksi yang dikirimkan oleh klien ke dalam sistem. Kebutuhan masukan meliputi:

- 1) Pengguna memasukkan kredensial autentikasi berupa alamat surel dan kata sandi untuk mengakses antarmuka demonstrasi.
- 2) Pengguna memasukkan instruksi pembuatan direktori baru beserta nama direktori yang diinginkan sebagai metadata logis.
- 3) Pengguna mengirimkan aliran data berkas cadangan melalui halaman demonstrasi unggah berkas.
- 4) Pengguna menentukan lokasi metadata berkas, baik pada halaman utama Berkas Saya maupun pada direktori tertentu.
- 5) Pengguna mengirimkan instruksi demonstrasi *storage*, seperti unggah, unduh, pratinjau metadata keamanan, pemindahan ke sampah secara logis, pemulihan item, dan penandaan item berbintang.

b. Kebutuhan Proses

Sistem menjalankan serangkaian proses komputasi terhadap data masukan yang diterima. Daftar kebutuhan proses mencakup:

- 1) Sistem memvalidasi kredensial akun dan mengelola sesi autentikasi pengguna.
- 2) Sistem mencatat identitas pemilik berkas, metadata berkas, serta lokasi metadata pada halaman utama Berkas Saya atau direktori tertentu.
- 3) Sistem mengelola struktur direktori dan hubungan hierarki antar direktori pada PostgreSQL.
- 4) Sistem memproses unggah berkas dari antarmuka demonstrasi menuju *Vault Core*.
- 5) Sistem mencegah penggunaan nama berkas aktif yang sama pada lokasi metadata yang sama.
- 6) Sistem memecah aliran data berkas menjadi *chunk* menggunakan FastCDC dan menghitung *hash* BLAKE3 untuk setiap *chunk*.
- 7) Sistem menyimpan *chunk* baru, menggunakan ulang *chunk* yang sudah tersedia, dan mencatat *manifest* objek pada BadgerDB di *Environment B*.
- 8) Sistem memproses *retrieval* dengan membaca *manifest* dan *chunk* melalui mekanisme baca terbatas sesuai rancangan *retrieval*.
- 9) Sistem memproses retensi dan *soft delete* sebagai penandaan logis pada metadata aplikasi tanpa menghapus *chunk* fisik, *manifest* objek, atau referensi penyimpanan pada *Vault Core*.
- 10) Sistem menampilkan metadata demonstratif berupa status *immutable*, nilai *hash*, ukuran objek, dan informasi deduplikasi.

c. Kebutuhan Luaran

Kebutuhan luaran mendefinisikan hasil pemrosesan yang dikembalikan oleh sistem kepada pengguna. Kebutuhan luaran meliputi:

- 1) Sistem menyajikan antarmuka demonstrasi Berkas Saya yang memuat daftar berkas dan direktori milik pengguna.
- 2) Sistem menampilkan metadata dasar berkas, seperti nama, ukuran, lokasi logis, dan waktu unggah.

- 3) Sistem menampilkan panel pratinjau keamanan yang memuat status penguncian objek *immutable*, nilai *hash*, dan referensi *manifest*.
- 4) Sistem mengembalikan berkas secara utuh ke penyimpanan lokal pengguna saat proses unduh berhasil dilakukan.
- 5) Sistem menampilkan status *soft delete* dan pemulihan item sebagai perubahan logis pada metadata aplikasi.
- 6) Sistem menampilkan informasi deduplikasi, seperti jumlah *chunk*, *chunk* baru, *chunk* yang digunakan ulang, dan rasio efisiensi penyimpanan.

3.4.2 Kebutuhan Nonfungsional

Kebutuhan nonfungsional mencakup spesifikasi lingkungan kerja yang mendukung jalannya aplikasi secara optimal.

a. Kebutuhan Perangkat Lunak

Penulis menggunakan beberapa perangkat lunak untuk mengembangkan dan menjalankan sistem:

- 1) Sistem Operasi: Windows 11 dengan Windows Subsystem for Linux (WSL2).
- 2) Bahasa Pemrograman dan Runtime: Go versi 1.21 dan Node.js versi 25.9.0.
- 3) Kerangka Kerja Aplikasi Web: React
- 4) Basis Data: BadgerDB dan PostgreSQL.
- 5) Alat Pengujian: Postman API Platform dan Chrome Browser.

b. Kebutuhan Perangkat Keras

Sistem membutuhkan spesifikasi perangkat keras minimum untuk menjamin kelancaran komputasi kriptografi:

- 1) Prosesor (CPU): Prosesor quad-core (contoh: Intel Core i5 atau AMD Ryzen 5).
- 2) Memori (RAM): Minimal 8 Gigabita (GB) untuk mendukung konkurensi data memori.
- 3) Media Penyimpanan: Solid State Drive (SSD) dengan kapasitas ruang minimal 50 GB guna mendukung kecepatan operasi baca-tulis BadgerDB.

c. Kebutuhan Keamanan Penyimpanan

Sistem membutuhkan rancangan keamanan penyimpanan untuk membatasi dampak kompromi pada lapisan aplikasi. Kebutuhan keamanan penyimpanan meliputi:

- 1) API Service tidak memiliki izin untuk menghapus atau menimpa *chunk* fisik yang telah tersimpan pada direktori *Vault Core*.
- 2) *Vault Core* hanya menerima operasi penyimpanan dan pembacaan sesuai rancangan, serta tidak menyediakan operasi penghapusan fisik pada prototipe.
- 3) Komunikasi antara API Service dan *Vault Core* dilakukan melalui jalur lokal terbatas menggunakan *Unix Domain Socket* (UDS).
- 4) Proses retrieval menggunakan akses baca terbatas atau *read-only mount* agar lapisan API dapat membaca *chunk* tanpa memperoleh izin modifikasi terhadap penyimpanan fisik.
- 5) Operasi *soft delete* dan retensi hanya mengubah metadata logis pada PostgreSQL tanpa menghapus *manifest* objek, referensi *chunk*, atau data fisik pada *Vault Core*.

3.4.3 Perancangan Konseptual dan Fisik

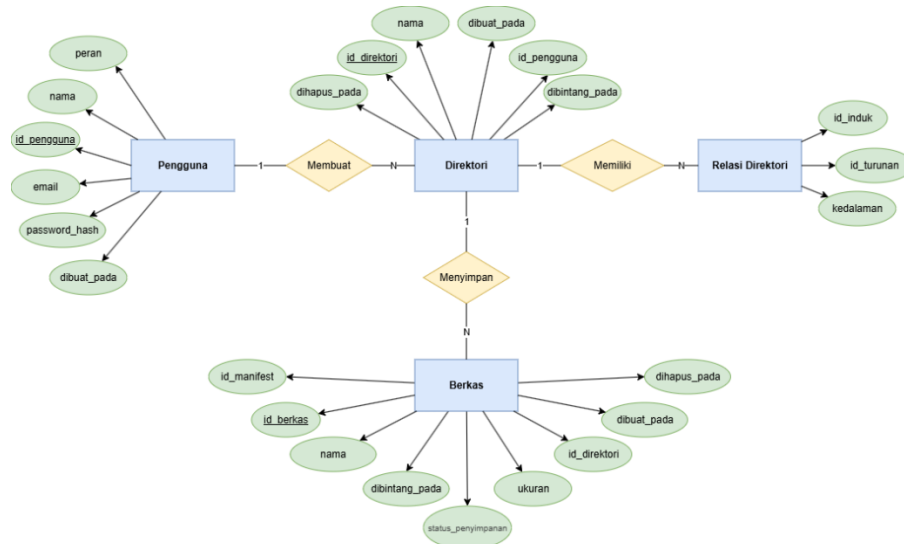
Penulis menyusun rancangan konseptual untuk memvisualisasikan alur informasi sistem dan rancangan fisik untuk mendokumentasikan struktur penyimpanan data. Rancangan konseptual mencakup *Entity Relationship Diagram* (ERD), *Data Flow Diagram* (DFD) dari Level 0 hingga Level 2, Flowchart, serta *wireframe* antarmuka pengguna berbasis web. Rancangan fisik mendokumentasikan skema basis data PostgreSQL dan struktur *key-value* BadgerDB yang menjadi fondasi penyimpanan metadata sistem.

3.4.3.1 Perancangan Konseptual

a. Entity Relationship Diagram (ERD)

Gambar 3.3 menunjukkan *Entity Relationship Diagram* (ERD) yang merepresentasikan skema basis data relasional untuk sistem penyimpanan berkas. Penulis merancang struktur data yang terdiri dari empat entitas utama, yaitu entitas Pengguna, entitas Direktori, entitas Relasi Direktori, dan entitas Berkas. Penulis

tetap menggunakan pendekatan *Closure Table* pada entitas Relasi Direktori untuk mendukung struktur subdirektori dengan kedalaman fleksibel.



Gambar 3.3 Entity Relationship Diagram (ERD)

Entitas Pengguna menyimpan informasi kredensial dan identitas pemilik data. Entitas Pengguna memiliki relasi *one-to-many* terhadap entitas Direktori karena satu pengguna dapat memiliki banyak direktori. Entitas Pengguna juga memiliki relasi *one-to-many* terhadap entitas Berkas karena berkas dapat disimpan langsung pada halaman utama Berkas Saya tanpa harus berada di dalam direktori tertentu.

Entitas Pengguna perlu ditambahkan atribut peran untuk membedakan hak akses pengguna biasa dan admin. Peran pengguna biasa digunakan untuk mengelola berkas dan direktori pribadi, sedangkan peran admin digunakan untuk memantau analitik aplikasi secara agregat. Admin tidak memiliki hak untuk membuka, mengubah, menghapus, memulihkan, atau melihat struktur berkas dan direktori milik pengguna.

Entitas Direktori berfungsi sebagai representasi folder logis pada lapisan aplikasi. Entitas Direktori tetap memiliki atribut `id_pengguna` sebagai penanda pemilik direktori. Entitas Direktori juga perlu ditambahkan atribut `dihapus_pada` untuk menandai status sampah dan atribut `dibintang_pada` untuk menandai status

Berbintang. Penambahan kedua atribut ini mendukung fitur sampah dan berbintang pada direktori tanpa menghapus struktur direktori secara langsung.

Entitas Relasi Direktori menyimpan pemetaan hierarki folder secara eksplisit. Entitas Relasi Direktori menghubungkan dua identitas dari entitas Direktori, yaitu `id_induk` dan `id_turunan`. Dengan menyimpan seluruh jalur hubungan antar direktori, sistem dapat melakukan pencarian isi subdirektori tanpa melakukan kueri berulang terhadap relasi rekursif.

Entitas Berkas menyimpan metadata dokumen yang diunggah oleh pengguna ke dalam sistem. Entitas Berkas perlu memiliki atribut `id_pengguna` sebagai penanda kepemilikan langsung terhadap pengguna. Entitas Berkas tetap memiliki atribut `id_direktori`, tetapi atribut tersebut bersifat opsional. Nilai `id_direktori` yang kosong menunjukkan bahwa berkas berada pada halaman utama Berkas Saya, sedangkan nilai `id_direktori` yang terisi menunjukkan bahwa berkas berada di dalam direktori tertentu.

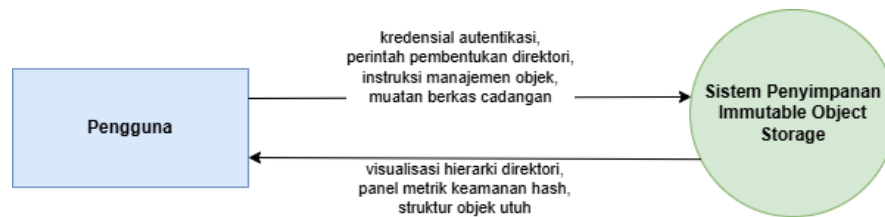
Entitas Berkas tetap menyimpan atribut `id_manifest` sebagai penghubung antara metadata berkas pada API dan *Object Manifest* di dalam *Vault Core* atau BadgerDB. Entitas Berkas juga menggunakan atribut `dihapus_pada` untuk menandai status Sampah dan atribut `dibintangi_pada` untuk menandai status Berbintang. Dengan rancangan ini, sistem dapat membedakan berkas aktif, berkas berbintang, dan berkas yang berada di Sampah tanpa langsung menghapus referensi manifest fisik dari *Vault Core*.

b. Data Flow Diagram (DFD)

Penulis merancang *Data Flow Diagram* (DFD) untuk memetakan alur data pada sistem *Immutable Object Storage* secara bertingkat dari Level 0 hingga Level 2. DFD Level 0 menggambarkan interaksi sistem secara keseluruhan, DFD Level 1 mengurai proses-proses utama, dan DFD Level 2 merinci setiap sub-proses secara lebih mendalam.

1) DFD Level 0 (Context Diagram)

DFD Level 0 digunakan untuk menggambarkan hubungan antara entitas eksternal dan sistem secara umum. Gambar 3.4 menyajikan DFD Level 0 sistem *Immutable Object Storage*.

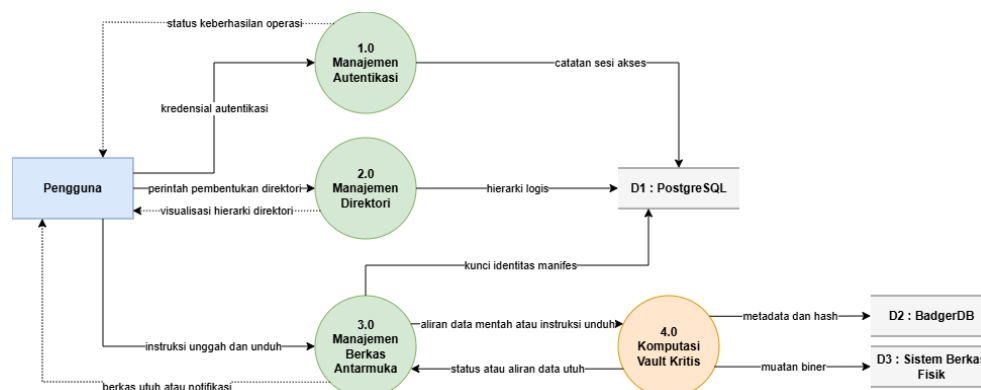


Gambar 3.4 Data Flow Diagram (DFD) Level 0

Gambar 3.4 menunjukkan Data Flow Diagram (DFD) Level 0 atau Diagram Konteks. Entitas eksternal pengguna berinteraksi secara dua arah dengan sistem penyimpanan *Immutable Object Storage*. Pengguna mengirimkan aliran data masukan berupa kredensial autentikasi, perintah pembentukan direktori, instruksi manajemen objek, dan muatan berkas cadangan. Sistem merespons aliran data masukan dengan mengembalikan visualisasi hierarki direktori pada dasbor web, panel metrik keamanan *hash*, serta struktur objek utuh saat operasi unduhan berlangsung.

2) DFD Level 1

DFD Level 1 digunakan untuk menguraikan proses utama yang terdapat pada sistem *Immutable Object Storage*. Gambar 3.5 menyajikan dekomposisi proses utama sistem ke dalam beberapa proses fungsional.



Gambar 3.5 Data Flow Diagram (DFD) Level 1

Gambar 3.5 menunjukkan *Data Flow Diagram* (DFD) Level 1 yang menjabarkan dekomposisi proses komputasi utama sistem. Sistem mendistribusikan aliran data ke dalam empat proses fungsional inti, yakni proses

manajemen autentikasi (1.0), proses manajemen direktori (2.0), proses manajemen berkas antarmuka (3.0), dan proses komputasi *Vault* kritis (4.0).

Proses 1.0 Manajemen Autentikasi memvalidasi identitas pengguna dan mencatat sesi akses ke dalam penyimpanan data PostgreSQL. Proses 2.0 Manajemen Direktori menerima perintah pembentukan tata letak folder dan menyimpan hierarki logis ke dalam penyimpanan data PostgreSQL.

Proses 3.0 Manajemen Berkas Antarmuka bertindak sebagai gerbang operasional utama yang menerima instruksi unggah dan instruksi unduh dari pengguna. Pada mekanisme pengunggahan, proses 3.0 Manajemen Berkas Antarmuka meneruskan aliran data mentah menuju proses 4.0 Komputasi *Vault* Kritis yang berada di dalam lingkungan terisolasi. Proses 4.0 Komputasi *Vault* Kritis mengeksekusi algoritma FastCDC untuk memecah data dan algoritma BLAKE3 untuk menghasilkan nilai kriptografi. Selanjutnya, proses 4.0 Komputasi *Vault* Kritis menyimpan *chunk* baru ke dalam Sistem Berkas Fisik setelah proses *hash* dan pemeriksaan deduplikasi selesai dilakukan.

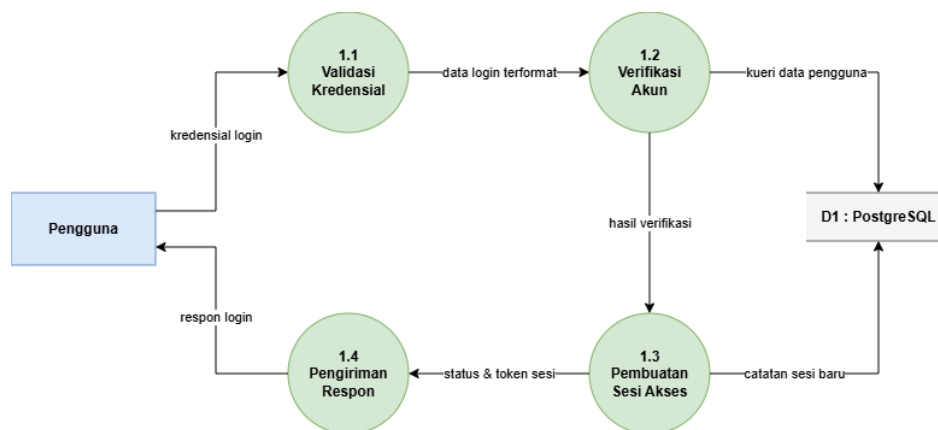
Sebagai tahap akhir sinkronisasi hibrida pada proses unggah, proses 4.0 Komputasi *Vault* Kritis mengembalikan kunci identitas manifes kepada proses 3.0 Manajemen Berkas Antarmuka. Proses 3.0 Manajemen Berkas Antarmuka menyimpan kunci identitas manifes ke dalam PostgreSQL sebagai jembatan penunjuk referensi data. Sebaliknya pada mekanisme pengunduhan, proses 4.0 Komputasi *Vault* Kritis melakukan rekonstruksi objek dengan mengambil muatan fisik dari Sistem Berkas Fisik, lalu mengembalikan aliran data utuh kepada proses 3.0 Manajemen Berkas Antarmuka untuk disajikan kembali kepada pengguna.

3) DFD Level 2

DFD Level 2 digunakan untuk merinci proses utama yang telah ditampilkan pada DFD Level 1. Setiap proses utama dijabarkan menjadi beberapa sub-proses agar alur data sistem dapat dipahami secara lebih rinci.

a) DFD Level 2 Proses 1 (Manajemen Autentikasi)

DFD Level 2 Proses 1 menggambarkan rincian alur kerja manajemen autentikasi pengguna. Gambar 3.6 menyajikan sub-proses yang terlibat dalam validasi identitas dan pembentukan sesi akses.

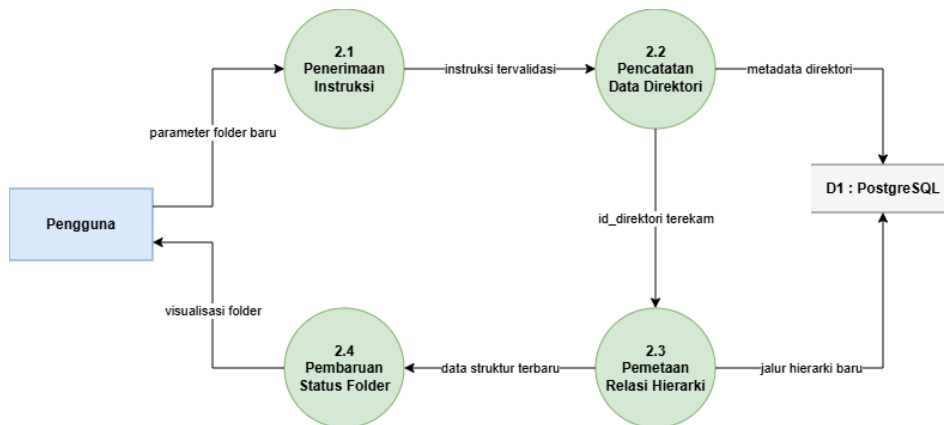


Gambar 3.6 Data Flow Diagram (DFD) Level 2 Proses 1

Gambar 3.6 menunjukkan DFD Level 2 Proses 1 yang menguraikan rincian alur kerja pada bagian Manajemen Autentikasi. Penulis membagi Proses 1 ke dalam empat sub-proses utama guna meningkatkan keamanan akses pengguna sebelum melakukan interaksi dengan sistem penyimpanan. Tahapan autentikasi melibatkan verifikasi identitas serta pengelolaan sesi aktif pengguna secara sistematis. Sub-proses 1.1 Validasi Kredensial menerima masukan berupa nama pengguna dan kata sandi dari entitas Pengguna. Selanjutnya, sub-proses 1.2 Verifikasi Akun melakukan kueri ke dalam tabel pengguna pada basis data PostgreSQL untuk mencocokkan nilai *hash* kata sandi. Sub-proses 1.3 Pembuatan Sesi Akses membentuk token akses unik setelah sistem berhasil memvalidasi identitas pengguna. Tahap akhir dilakukan oleh sub-proses 1.4 Pengiriman Respons yang meneruskan status keberhasilan *login* dan token sesi kembali kepada pengguna sebagai bukti akses untuk menggunakan fitur sistem lainnya.

b) DFD Level 2 Proses 2 (Manajemen Direktori)

DFD Level 2 Proses 2 menggambarkan rincian pengelolaan direktori pada sistem. Gambar 3.7 menyajikan alur data pembentukan direktori dan pemetaan relasi hierarki folder.

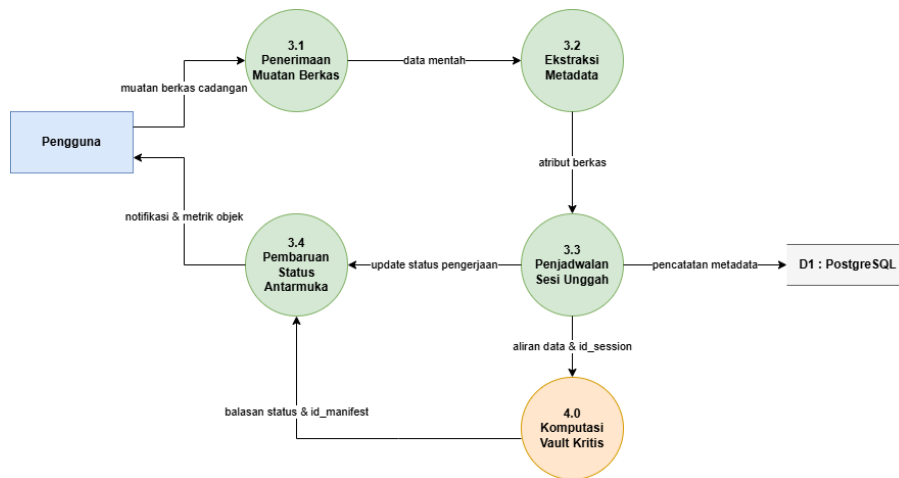


Gambar 3.7 Data Flow Diagram (DFD) Level 2 Proses 2

Gambar 3.7 menunjukkan DFD Level 2 Proses 2 yang menguraikan rincian mekanisme pengelolaan struktur folder di dalam sistem. Penulis merancang empat sub-proses utama untuk menangani pembentukan direktori serta pemetaan hierarki menggunakan pendekatan *Closure Table*. Sub-proses 2.1 Penerimaan Instruksi Direktori menerima parameter nama folder dan identitas folder induk dari entitas Pengguna. Selanjutnya, sub-proses 2.2 Pencatatan Data Direktori menyimpan metadata dasar folder ke dalam tabel direktori pada basis data PostgreSQL. Sub-proses 2.3 Pemetaan Relasi Hierarki menjalankan logika untuk mendaftarkan hubungan antara folder baru dengan seluruh leluhurnya di dalam tabel relasi direktori guna mendukung struktur folder bersarang dengan kedalaman yang fleksibel. Tahap akhir dilakukan oleh sub-proses 2.4 Pembaruan Status Folder yang mengirimkan konfirmasi keberhasilan serta data struktur terbaru kembali kepada Pengguna untuk memperbarui tampilan antarmuka.

c) DFD Level 2 Proses 3 (Manajemen Berkas Antarmuka)

DFD Level 2 Proses 3 menggambarkan rincian manajemen berkas pada lapisan antarmuka sistem. Gambar 3.8 menyajikan alur penerimaan berkas, ekstraksi metadata, penjadwalan unggah, dan pembaruan status antarmuka.

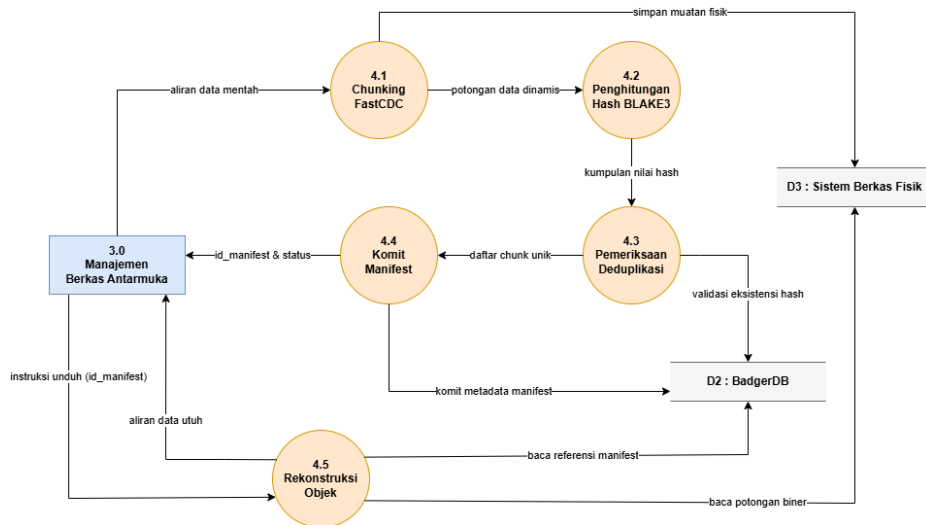


Gambar 3.8 Data Flow Diagram (DFD) Level 2 Proses 3

Gambar 3.8 menunjukkan DFD Level 2 Proses 3 yang menguraikan rincian mekanisme manajemen berkas pada lapisan antarmuka sistem. Penulis merancang empat sub-proses utama untuk menjembatani interaksi antara entitas Pengguna dengan lapisan komputasi kritis di dalam lingkungan *Vault Core*. Sub-proses 3.1 Penerimaan Muatan Berkas menerima aliran data berkas cadangan dari Pengguna melalui protokol HTTP. Selanjutnya, sub-proses 3.2 Ekstraksi Metadata melakukan identifikasi awal terhadap karakteristik berkas seperti nama, ukuran, dan tipe berkas. Sub-proses 3.3 Penjadwalan Sesi Unggah membentuk identitas sesi sementara dan mendaftarkan metadata awal ke dalam tabel berkas pada basis data PostgreSQL. Tahap akhir dilakukan oleh sub-proses 3.4 Pembaruan Status Antarmuka yang mengirimkan notifikasi progres serta hasil identifikasi objek kembali kepada Pengguna setelah menerima umpan balik dari Proses 4.0.

d) DFD Level 2 Proses 4 (Manajemen Vault Kritis)

DFD Level 2 Proses 4 menggambarkan rincian proses komputasi kritis yang berlangsung di dalam *Vault Core*. Gambar 3.9 menyajikan alur *chunking*, *hashing*, deduplikasi, komit *manifest*, dan rekonstruksi objek.



Gambar 3.9 Data Flow Diagram (DFD) Level 2 Proses 4

Gambar 3.9 menunjukkan DFD Level 2 Proses 4 yang menguraikan mekanisme komputasi kritis di dalam Vault Core. Sub-proses 4.1 Chunking FastCDC menerima aliran data mentah dan memecahnya menjadi *chunk* berukuran dinamis. Sub-proses 4.2 Penghitungan Hash BLAKE3 menghasilkan identitas berbasis *hash* untuk setiap *chunk*. Sub-proses 4.3 Pemeriksaan Deduplikasi memeriksa keberadaan *hash* pada D2: BadgerDB. Jika *hash* belum tersedia, sistem menyimpan *chunk* baru ke D3: Sistem Berkas Fisik. Jika *hash* sudah tersedia, sistem menggunakan referensi *chunk* yang telah ada. Sub-proses 4.4 Komit Manifest mencatat daftar *chunk* penyusun objek ke dalam BadgerDB. Sub-proses 4.5 Rekonstruksi Objek menyediakan referensi *manifest* dan daftar *chunk* yang diperlukan untuk proses unduh. API Service kemudian membaca *chunk* melalui akses *read-only mount*, merakit objek sesuai urutan *manifest*, dan mengembalikan berkas utuh kepada pengguna tanpa memperoleh izin tulis atau hapus terhadap penyimpanan fisik.

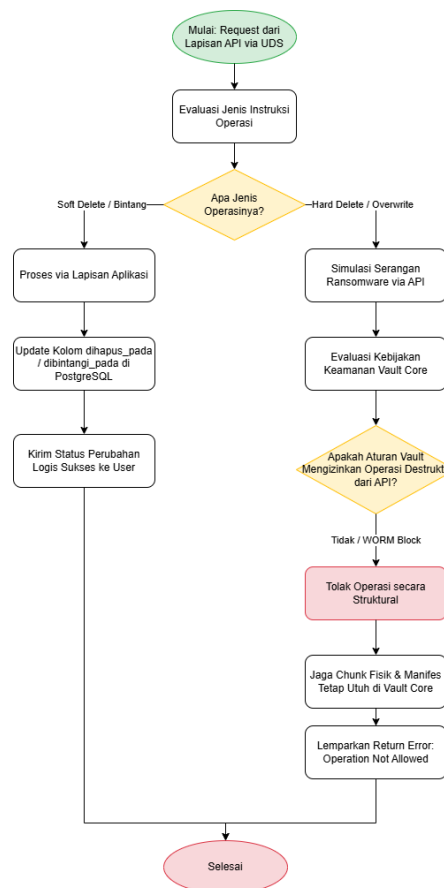
DFD Level 2 Proses 4 tidak menyediakan aliran data untuk penghapusan fisik *chunk* dari lapisan API menuju Vault Core. Operasi penghapusan pada antarmuka pengguna hanya diproses sebagai *soft delete* pada metadata aplikasi. *Chunk* fisik, manifest objek, dan referensi penyimpanan tetap berada di Vault Core

sehingga manipulasi melalui jalur aplikasi tidak langsung menghapus data cadangan yang telah tersimpan.

c. Flowchart

1) Flowchart Validasi Operasi Destruktif terhadap Vault Core

Flowchart validasi operasi destruktif terhadap Vault Core digunakan untuk menggambarkan mekanisme pembatasan operasi yang berpotensi merusak data fisik pada sistem Immutable Object Storage. Flowchart ini menjelaskan perbedaan perlakuan antara operasi logis yang masih diperbolehkan oleh lapisan aplikasi, seperti soft delete dan penandaan berbintang, dengan operasi destruktif seperti penghapusan fisik dan penimpaan objek. Pemisahan alur tersebut diperlukan agar perubahan status berkas tetap dapat dilakukan pada metadata aplikasi tanpa memberikan hak destruktif terhadap chunk fisik, manifest objek, maupun referensi penyimpanan di Vault Core.



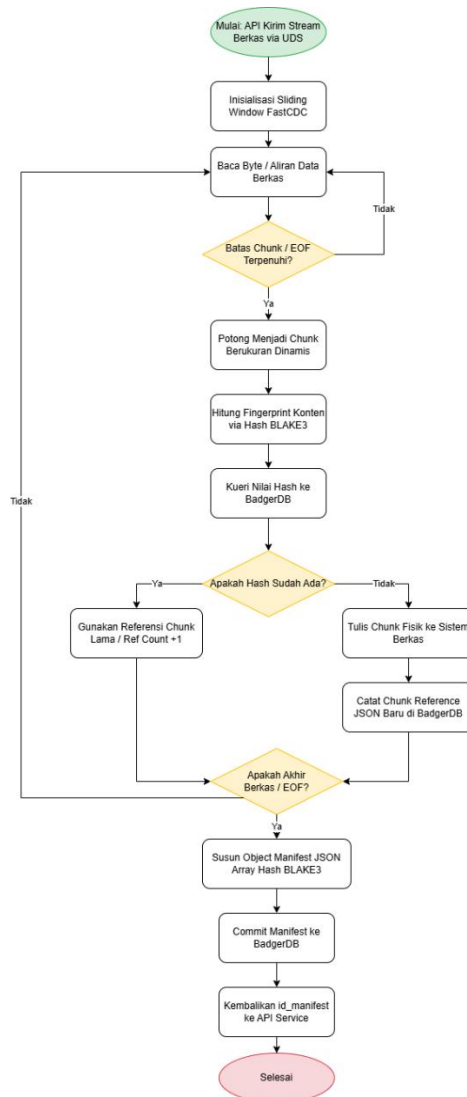
Gambar 3.10 Flowchart Validasi Operasi Destruktif terhadap Vault Core

Gambar 3.10 menunjukkan flowchart validasi operasi destruktif terhadap Vault Core. Proses dimulai ketika API Service mengirimkan permintaan operasi melalui Unix Domain Socket (UDS). Sistem kemudian mengevaluasi jenis operasi yang diterima. Apabila operasi yang diminta berupa soft delete atau penandaan berbintang, proses hanya dilakukan pada lapisan aplikasi dengan memperbarui metadata logis pada PostgreSQL, seperti kolom `dihapus_pada` atau `dibintangi_pada`. Setelah metadata berhasil diperbarui, sistem mengirimkan status perubahan logis kepada pengguna.

Apabila operasi yang diterima berupa penghapusan fisik atau penimpaan objek, sistem memperlakukan permintaan tersebut sebagai operasi destruktif terhadap Vault Core. Permintaan tersebut kemudian divalidasi berdasarkan kebijakan keamanan Vault Core. Karena sistem dirancang dengan prinsip immutable, operasi destruktif dari API Service tidak diberikan izin untuk menghapus atau menimpa chunk fisik maupun manifest objek. Dengan demikian, Vault Core menolak operasi tersebut secara struktural, menjaga chunk fisik dan manifest tetap utuh, serta mengembalikan pesan galat `Operation Not Allowed`.

2) Flowchart Proses Unggah dan Deduplikasi Berkas.

Flowchart proses unggah dan deduplikasi berkas digunakan untuk menggambarkan urutan kerja penyimpanan objek pada lapisan Vault Core secara lebih rinci. Flowchart ini menjelaskan tahapan fisik yang terjadi setelah API Service menerima berkas dari pengguna dan meneruskannya ke Vault Core melalui Unix Domain Socket (UDS). Tahapan tersebut meliputi pembacaan aliran data berkas, pembentukan chunk menggunakan FastCDC, penghitungan hash BLAKE3, pemeriksaan deduplikasi pada BadgerDB, penulisan chunk baru ke sistem berkas fisik, serta pembentukan manifest objek sebagai referensi rekonstruksi berkas.

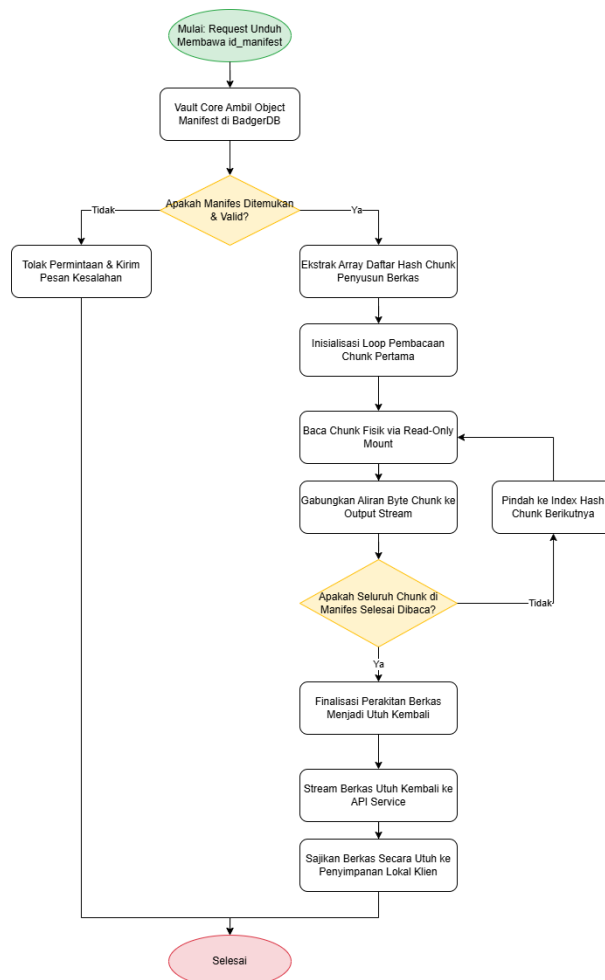


Gambar 3.11 Flowchart Proses Unggah dan Deduplikasi Berkas

Gambar 3.11 menunjukkan proses unggah dan deduplikasi berkas pada Vault Core. Berkas yang diterima dari API Service dipotong menjadi *chunk* menggunakan FastCDC, kemudian setiap *chunk* dihitung hash-nya menggunakan BLAKE3. *Hash* tersebut diperiksa pada BadgerDB untuk menentukan apakah *chunk* sudah pernah tersimpan. Jika sudah ada, sistem menggunakan referensi *chunk* lama. Jika belum ada, sistem menyimpan *chunk* baru dan mencatat referensinya. Setelah seluruh *chunk* diproses, *Vault Core* menyusun manifest objek dan mengembalikan *id_manifest* ke API Service.

3) Flowchart Proses Rekonstruksi dan Pengunduhan Objek

Flowchart proses rekonstruksi dan pengunduhan objek digunakan untuk menggambarkan tahapan sistem dalam mengembalikan berkas utuh berdasarkan manifest objek yang telah tersimpan. Flowchart ini menjelaskan proses pembacaan metadata manifest dari BadgerDB, pengambilan daftar *hash chunk* penyusun berkas, pembacaan *chunk* fisik melalui *read-only mount*, serta penggabungan kembali aliran *byte chunk* sesuai urutan manifest. Proses ini diperlukan agar berkas yang sebelumnya disimpan dalam bentuk *chunk* dapat direkonstruksi kembali tanpa memberikan hak tulis atau hapus terhadap penyimpanan fisik.



Gambar 3.12 Flowchart Proses Rekonstruksi dan Pengunduhan Objek

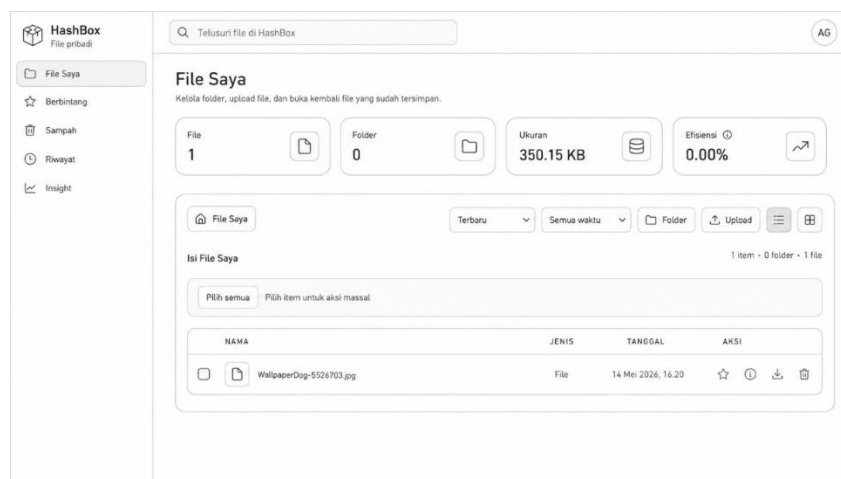
Gambar 3.12 menunjukkan flowchart proses rekonstruksi dan pengunduhan objek menggambarkan tahapan sistem dalam mengembalikan berkas utuh

berdasarkan id_manifest. Vault Core mengambil manifest objek dari BadgerDB, memvalidasi keberadaannya, kemudian membaca daftar hash chunk penyusun berkas. Setiap chunk dibaca secara berurutan melalui read-only mount dan digabungkan kembali menjadi output stream. Apabila seluruh chunk telah selesai dibaca, sistem mengirimkan berkas utuh ke API Service untuk disajikan kepada pengguna. Alur ini memastikan proses unduh berjalan tanpa memberikan hak tulis atau hapus terhadap penyimpanan fisik.

d. Wireframe

Sistem *Immutable Object Storage* menyediakan antarmuka grafis berbasis web sebagai media demonstrasi untuk menjalankan alur penyimpanan, pengunduhan, *soft delete*, serta penampilan metadata keamanan objek. Penulis menyusun *wireframe* untuk memvisualisasikan interaksi pengguna dengan sistem tanpa menjadikan antarmuka sebagai fokus utama evaluasi penelitian. Rancangan visual diarahkan untuk memperlihatkan hubungan antara metadata aplikasi, status *immutable*, nilai *hash*, *manifest* objek, dan informasi deduplikasi yang dihasilkan oleh *Vault Core*.

4) Wireframe Halaman Dasbor

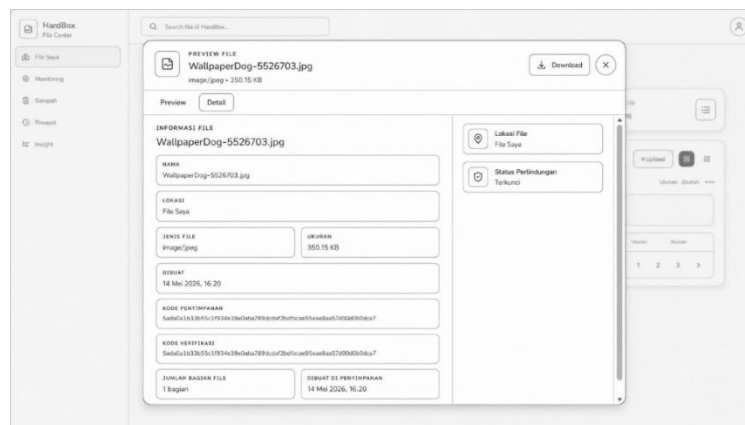


Gambar 3.13 Wireframe Halaman Dasbor

Gambar 3.13 menunjukkan *wireframe* antarmuka halaman dasbor utama berbasis web. Halaman dasbor berfungsi sebagai ruang kerja bagi pengguna untuk mengelola dokumen cadangan. Sistem menampilkan daftar berkas yang telah

diunggah beserta informasi metadata dasar seperti nama berkas, ukuran, dan tanggal unggah. Pengguna mengunggah berkas baru melalui tombol aksi utama yang memicu jendela pemilihan berkas. Antarmuka web mengirimkan aliran data menuju lapisan API. Lapisan API kemudian meneruskan aliran data tersebut ke *Vault Core* untuk diproses melalui FastCDC, *hashing* BLAKE3, deduplikasi, dan penyimpanan *chunk*. Metadata awal disimpan dengan status sementara sampai Vault Core mengembalikan *id_manifest* sebagai tanda bahwa proses penyimpanan objek berhasil.

5) Wireframe Halaman Pratinjau Berkas



Gambar 3.14 Wireframe Halaman Pratinjau Berkas

Gambar 3.14 menunjukkan *wireframe* antarmuka halaman rincian berkas. Halaman rincian berkas menyajikan informasi spesifik mengenai objek yang tersimpan, termasuk nilai *hash* kriptografi BLAKE3 dan status retensi dari *Vault Core*. Sistem memvisualisasikan indikator penguncian objek secara grafis untuk menunjukkan bahwa berkas berstatus *immutable* dan memiliki perlindungan terhadap operasi manipulasi yang tidak sesuai dengan rancangan sistem. Sistem juga menyediakan tombol unduh bagi pengguna untuk mengambil kembali berkas cadangan secara utuh ke penyimpanan lokal.

3.4.3.2 Perancangan Fisik

Perancangan fisik basis data mendokumentasikan spesifikasi teknis dari tabel PostgreSQL dan struktur *key-value* BadgerDB yang digunakan dalam pengembangan sistem. PostgreSQL digunakan untuk mengelola metadata

relasional pada *Environment A*, sedangkan BadgerDB digunakan untuk menyimpan metadata *chunk* dan *manifest* pada *Vault Core*.

a. Basis Data Relasional

1) Tabel Pengguna

Tabel 3.2 menyajikan struktur entitas pengguna. Entitas pengguna menyimpan kredensial autentikasi klien yang mengakses aplikasi web.

Tabel 3.2 Tabel Pengguna

No	Nama Kolom	Tipe Data	Keterangan
1	id_pengguna	UUID	<i>Primary Key</i> , identitas unik pengguna.
2	nama	VARCHAR	Nama lengkap pengguna aplikasi.
3	email	VARCHAR	Alamat surel (<i>email</i>) pengguna untuk keperluan <i>login</i> .
4	password_hash	VARCHAR	Nilai hash kata sandi pengguna yang dibuat menggunakan algoritma hashing aman dan salt.
5	peran	ENUM	Menyimpan jenis hak akses akun dalam sistem
6	dibuat_pada	TIMESTAMP	Waktu pembuatan akun pengguna.

2) Tabel Direktori

Tabel 3.3 menyajikan struktur entitas direktori pada lapisan aplikasi. Entitas direktori berfungsi untuk mengelola struktur folder logis milik pengguna pada lapisan aplikasi.

Tabel 3.3 Tabel Direktori

No	Nama Kolom	Tipe Data	Keterangan
1	id_direktori	UUID	<i>Primary Key</i> , identitas unik direktori.
2	id_pengguna	UUID	<i>Foreign Key</i> , merujuk pada identitas pemilik di tabel Pengguna.
3	nama	VARCHAR	Nama direktori yang dibuat oleh pengguna.
4	dibintangi_pada	TIMESTAMP	Waktu menandai bintang direktori.
5	dibuat_pada	TIMESTAMP	Waktu pembuatan direktori.

No	Nama Kolom	Tipe Data	Keterangan
6	dihapus_pada	TIMESTAMP	Waktu penghapusan direktori.

3) Tabel Relasi Direktori

Tabel 3.4 menyajikan struktur entitas relasi direktori. Tabel relasi direktori mengimplementasikan logika tabel penutup dengan menggunakan *id_induk* sebagai referensi induk dalam struktur hierarki.

Tabel 3.4 Tabel Relasi Direktori

No	Nama Kolom	Tipe Data	Keterangan
1	id_induk	UUID	Primary Key dan Foreign Key yang merujuk pada direktori <i>parent</i> .
2	id_turunan	UUID	Primary Key dan Foreign Key yang merujuk pada direktori turunan.
3	kedalaman	INTEGER	Jarak hierarki antara direktori <i>parent</i> dan direktori turunan.

4) Tabel Berkas

Tabel 3.5 menyajikan struktur entitas berkas. Tabel berkas menyimpan seluruh informasi dokumen yang dikelola oleh sistem pada lokasi logis, baik di halaman utama Berkas Saya maupun di dalam direktori tertentu.

Tabel 3.5 Tabel Berkas

No	Nama Kolom	Tipe Data	Keterangan
1	id_berkas	UUID	<i>Primary Key</i> , identitas unik data berkas pada aplikasi web.
2	id_pengguna	UUID	<i>Foreign Key</i> , merujuk pada pengguna yang memiliki berkas.
3	id_direktori	UUID	<i>Foreign Key</i> , merujuk pada lokasi penyimpanan berkas di tabel Direktori.
4	nama	VARCHAR	Nama asli dokumen yang diunggah beserta ekstensinya.

Tabel 3.5 Tabel Berkas

No	Nama Kolom	Tipe Data	Keterangan
5	ukuran	BIGINT	Kapasitas ukuran berkas dalam satuan byte.
6	id_manifest	VARCHAR	Kunci referensi yang menghubungkan metadata berkas pada API dengan <i>Object Manifest</i> di dalam <i>Vault Core</i> atau BadgerDB.
7	status_penyimpanan	VARCHAR	Status proses penyimpanan berkas, seperti <i>pending</i> , <i>committed</i> , atau <i>failed</i> , untuk membedakan metadata berkas yang masih diproses, berhasil dikunci melalui <i>manifest</i> , atau gagal disimpan.
8	dibuat_pada	TIMESTAMP	Waktu pengunggahan berkas berhasil diselesaikan.
9	dihapus_pada	TIMESTAMP	Penanda <i>soft delete</i> untuk menyembunyikan berkas dari dasbor pengguna tanpa menghapus referensi manifes fisik di <i>Vault Core</i> .
10	dibintangi_pada	TIMESTAMP	Waktu menandai bintang berkas.

b. Basis Data Key-Value

Sistem mengimplementasikan basis data non-relasional berjenis *key-value* menggunakan BadgerDB di dalam lingkungan *Vault Core*. Penulis merancang skema *key-value* untuk mengelola metadata fisik berkas tanpa bergantung pada struktur tabel relasional. Basis data *key-value* mengelompokkan manajemen metadata ke dalam tiga struktur data utama secara terpisah.

1) Struktur Data Object Manifest

Struktur data pertama adalah *Object Manifest* yang bertugas mengelola metadata utama dokumen. Sistem menggunakan format kunci spesifik berupa

manifest:<file_hash> untuk memetakan urutan potongan data. Atribut JSON menyimpan daftar nilai *hash* BLAKE3 penyusun dokumen, total kapasitas keseluruhan berkas dalam satuan *byte*, serta stempel waktu saat sistem membentuk dan mengunci dokumen manifest.

2) Struktur Data Chunk Reference

Struktur data kedua adalah *Chunk Reference* yang berfungsi mengelola metadata fisik potongan data secara individual. Sistem menerapkan nilai *hash* kriptografi BLAKE3 sebagai kunci pencarian utama, sementara objek JSON menyimpan metrik untuk memfasilitasi algoritma deduplikasi otomatis. Atribut pada objek JSON mencakup ukuran kapasitas potongan data, jumlah dokumen manifest aktif yang merujuk pada potongan data spesifik, serta status penguncian retensi keamanan untuk melacak transisi perlindungan data.

3) Struktur Data Upload Session

Struktur data ketiga adalah *Upload Session* yang berperan memantau status pengunggahan berkas secara langsung. Sistem menjadikan identitas sesi sementara sebagai kunci pencarian untuk menjamin sifat idempoten dan mencegah duplikasi transmisi saat klien mengalami kegagalan jaringan. Objek JSON pada struktur sesi merekam daftar nilai *hash* dari potongan data yang berhasil sistem pindahkan ke area persinggahan, beserta batas waktu kedaluwarsa sebelum sistem mengeksekusi pembersihan otomatis terhadap data sementara.

Struktur *Upload Session* digunakan untuk mendukung konsistensi proses penyimpanan saat terjadi kegagalan unggah. Selama proses unggah belum selesai, sistem belum menetapkan objek sebagai *manifest* final. *Manifest* objek baru dianggap valid setelah seluruh *chunk* yang diperlukan berhasil diproses dan tahap *commit* selesai dilakukan. Dengan rancangan ini, kegagalan proses di tengah unggah tidak menghasilkan *metadata* berkas yang menunjuk pada objek tidak lengkap.

BAB IV PRODUK

4.1 Hasil

Penelitian menghasilkan prototipe sistem *Immutable Object Storage* yang digunakan untuk melindungi repositori cadangan dari manipulasi *ransomware*. Sistem dikembangkan dengan pendekatan *Content-Addressable Storage (CAS)* dan algoritma *Fast Content-Defined Chunking (FastCDC)* untuk mendukung penyimpanan data yang *immutable* dan efisien. Prototipe dirancang pada lingkungan *single-node* dengan pemisahan otoritas antara lapisan aplikasi dan lapisan penyimpanan.

Sistem dibangun menggunakan bahasa pemrograman Go pada komponen inti penyimpanan dan React pada antarmuka web demonstrasi. Lapisan aplikasi menggunakan PostgreSQL untuk menyimpan metadata pengguna, direktori, dan status berkas. Lapisan *Vault Core* menggunakan BadgerDB untuk menyimpan manifest objek dan referensi *chunk*. Setiap berkas yang diunggah diproses menjadi *chunk*, diberi identitas *hash* menggunakan BLAKE3, lalu disimpan dengan mekanisme deduplikasi.

Fitur utama prototipe mencakup autentikasi pengguna, pengelolaan direktori, unggah berkas, unduh berkas, pratinjau metadata keamanan, penandaan berkas berbintang, *soft delete*, dan pemulihan item dari sampah. Antarmuka web berfungsi sebagai media demonstrasi untuk memperlihatkan alur penyimpanan, status *immutable*, informasi *hash*, jumlah *chunk*, serta referensi manifest objek.

Pengujian sistem diarahkan untuk mengevaluasi fungsi utama, kemampuan deduplikasi, rekonstruksi objek, konsistensi data setelah kegagalan proses, dan ketahanan sistem terhadap skenario manipulasi melalui lapisan aplikasi. Hasil pengujian digunakan untuk menilai kemampuan deduplikasi, keberhasilan rekonstruksi objek, konsistensi data setelah kegagalan proses, dan ketahanan sistem terhadap skenario manipulasi melalui lapisan aplikasi. Pengujian tersebut menjadi dasar pembahasan untuk menunjukkan bahwa prototipe tidak hanya mampu menyimpan dan mengambil kembali berkas, tetapi juga membatasi operasi destruktif terhadap *Vault Core*.

4.2 Pembahasan Hasil

Pembahasan hasil menjelaskan cara kerja prototipe sistem *Immutable Object Storage* berdasarkan proses penyimpanan, tampilan antarmuka, dan pengujian sistem. Pengujian dilakukan untuk membandingkan respons sistem pada kondisi normal dan tidak normal. Kondisi normal mencakup proses unggah berkas baru, unggah berkas identik, unggah berkas dengan perubahan sebagian, deduplikasi, pengunduhan, *soft delete*, dan pemulihan objek sesuai alur sistem. Kondisi tidak normal mencakup berkas tidak valid, kegagalan proses unggah, permintaan *manifest* tidak valid, serta simulasi manipulasi melalui lapisan aplikasi.

4.2.1 Proses Penyimpanan dan Deduplikasi Objek

Proses penyimpanan diawali ketika pengguna mengunggah berkas melalui antarmuka web. API Service menerima berkas, membaca metadata dasar, lalu meneruskan aliran data menuju *Vault Core*. *Vault Core* memproses berkas menggunakan algoritma *Fast Content-Defined Chunking* (FastCDC) untuk membagi berkas menjadi beberapa *chunk* berdasarkan isi konten. Setelah proses pemecahan selesai, sistem menghitung nilai *hash* setiap *chunk* menggunakan BLAKE3.

Pada kondisi normal, sistem menyimpan *chunk* baru ke penyimpanan fisik apabila nilai *hash* belum ditemukan pada BadgerDB. Sistem kemudian membuat *manifest* objek yang berisi urutan *hash chunk* penyusun berkas. *Manifest* tersebut memungkinkan sistem merekonstruksi objek saat pengguna melakukan pengunduhan. Dengan mekanisme ini, sistem tidak menyimpan ulang keseluruhan berkas apabila sebagian *chunk* sudah tersedia pada repositori.

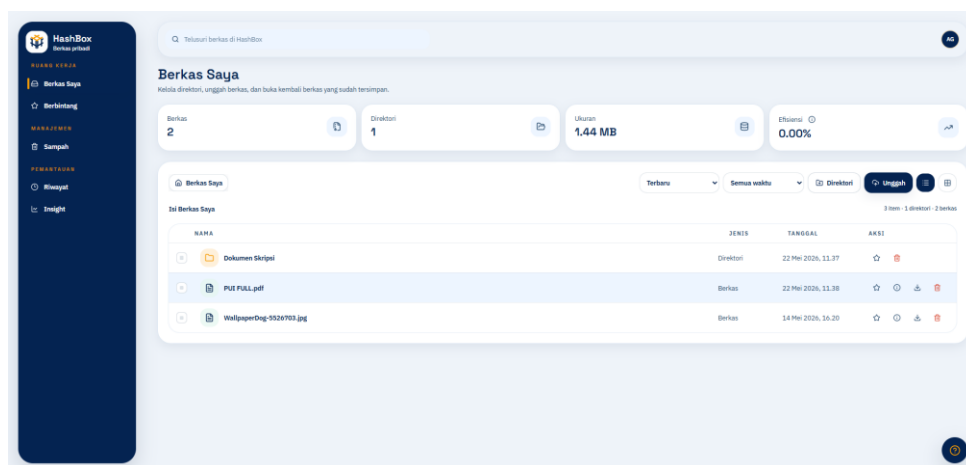
Pada kondisi berkas identik, sistem mendeteksi seluruh *chunk* sebagai data yang telah tersedia sehingga sistem hanya membuat *manifest* baru tanpa menulis ulang *chunk* fisik yang sama. Pada kondisi berkas mengalami perubahan sebagian, sistem menyimpan *chunk* baru hanya pada bagian yang berubah, sedangkan *chunk* yang tidak berubah tetap menggunakan referensi lama. Perbandingan ini menunjukkan bahwa CAS dan FastCDC dapat digunakan untuk menekan pemborosan kapasitas pada skenario pencadangan berulang.

Pada kondisi tidak normal, seperti unggah gagal atau koneksi terputus saat proses penyimpanan, sistem perlu menjaga konsistensi metadata agar *manifest* tidak menunjuk pada *chunk* yang belum lengkap. Jika proses belum mencapai tahap komit *manifest*, sistem tidak boleh mencatat objek sebagai berkas valid. Hasil pengujian kondisi kegagalan proses menunjukkan bahwa sistem tidak mencatat objek sebagai berkas valid sebelum tahap *commit manifest* selesai dilakukan.

4.2.2 User Interface

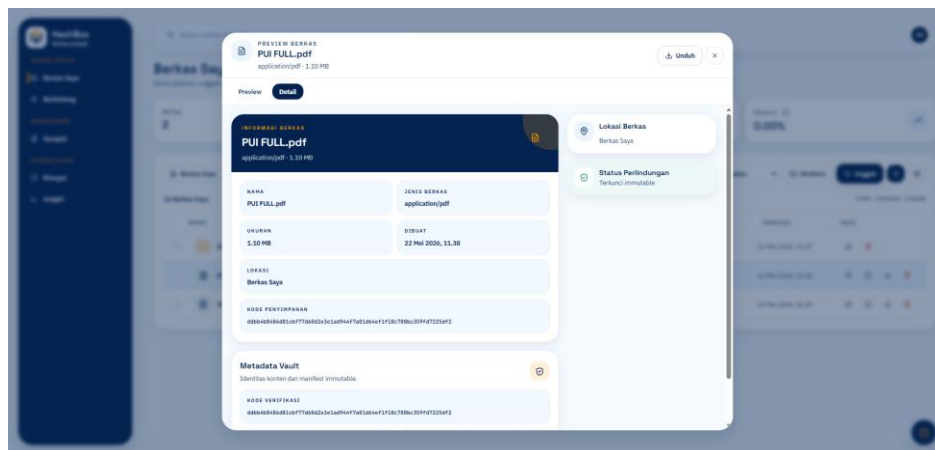
Antarmuka web berfungsi sebagai media demonstrasi untuk menjalankan dan memperlihatkan alur kerja sistem penyimpanan. Pembahasan antarmuka dibatasi pada fungsi yang berkaitan dengan proses unggah, unduh, *soft delete* logis, pemulihan item, serta penampilan metadata keamanan seperti status immutable, nilai *hash*, jumlah *chunk*, dan referensi *manifest* objek. Pengguna dapat mengunggah berkas, membuka direktori, melihat metadata dasar, menandai berkas berbintang, memindahkan berkas ke sampah secara logis, dan mengunduh kembali objek yang telah tersimpan.

Gambar 4.1 menampilkan halaman dasbor sistem. Halaman ini menyediakan tombol unggah berkas dan daftar objek cadangan yang telah tersimpan. Sistem menampilkan informasi seperti nama berkas, ukuran, waktu unggah, dan status objek. Tampilan ini membantu pengguna memastikan bahwa berkas telah masuk ke sistem penyimpanan.



Gambar 4.1 Dasbor Sistem

Gambar 4.2 menampilkan halaman rincian berkas. Halaman rincian berkas menyajikan informasi keamanan, seperti nilai *hash* objek, jumlah *chunk*, status *immutable*, dan referensi *manifest*. Informasi tersebut digunakan untuk memperlihatkan bahwa objek tidak hanya tersimpan sebagai berkas biasa, tetapi juga memiliki identitas berbasis konten dan metadata penyimpanan yang dapat diverifikasi.



Gambar 4.2 Rincian Berkas

Fitur *soft delete* pada antarmuka hanya mengubah status logis objek pada metadata aplikasi. Sistem tidak menghapus *chunk* fisik, *manifest* objek, atau referensi penyimpanan di *Vault Core*. Dengan demikian, penghapusan dari antarmuka tidak langsung menghilangkan data cadangan dari repositori inti.

4.2.3 Pengujian Sistem

Pengujian sistem dilakukan menggunakan dua pendekatan, yaitu pengujian *black box* dan pengujian keamanan. Pengujian *black box* digunakan untuk membandingkan kesesuaian keluaran sistem pada fungsi utama, meliputi unggah berkas baru, unggah berkas identik, unggah berkas dengan perubahan sebagian, pengunduhan, *soft delete*, dan pemulihan objek. Pengujian keamanan digunakan untuk memvalidasi pembatasan operasi destruktif terhadap *Vault Core* melalui skenario manipulasi *ransomware*. Kondisi tidak normal mencakup berkas tidak valid, kegagalan proses unggah, permintaan *manifest* tidak valid, serta percobaan penghapusan atau penimpaan objek melalui jalur aplikasi. Tabel 4.1 memuat hasil pengujian sistem yang terdiri dari skenario pengujian *black box* pada fungsi utama

dan skenario pengujian keamanan untuk memvalidasi pembatasan operasi destruktif terhadap Vault Core.

Tabel 4.1 Hasil Pengujian

No	Parameter Pengujian	Kondisi	Skenario Pengujian	Hasil yang Diharapkan	Hasil Uji
1	Unggah berkas	Normal	Pengguna mengunggah berkas valid	Sistem menerima berkas, membuat <i>chunk</i> , menghitung <i>hash</i> , dan mencatat <i>manifest</i>	Berhasil
2	Unggah berkas identik	Normal	Pengguna mengunggah berkas yang sama dua kali	Sistem menggunakan ulang <i>chunk</i> yang sudah tersedia dan tidak menulis ulang data fisik	Berhasil
3	Unggah berkas dengan perubahan sebagian	Normal	Pengguna mengunggah versi berkas yang hanya berubah pada sebagian isi	Sistem hanya menyimpan <i>chunk</i> baru pada bagian yang berubah	Berhasil
4	Pengunduhan berkas	Normal	Pengguna mengunduh	Sistem merekonstruksi objek	Berhasil

Tabel 4.1 Hasil Pengujian

No	Parameter Pengujian	Kondisi	Skenario Pengujian	Hasil yang Diharapkan	Hasil Uji
			objek yang tersimpan	berdasarkan <i>manifest</i> dan mengembalikan berkas utuh	
5	<i>Soft delete</i>	Normal	Pengguna memindahkan berkas ke sampah	Sistem mengubah status logis tanpa menghapus <i>chunk</i> fisik dan <i>manifest</i>	Berhasil
6	Pemulihan item	Normal	Pengguna memulihkan berkas dari sampah	Sistem mengembalikan status objek agar dapat diakses kembali	Berhasil
7	Berkas kosong atau tidak valid	Tidak normal	Pengguna mengunggah berkas kosong atau format masukan tidak valid	Sistem menolak permintaan dan mengirimkan pesan kesalahan	Berhasil
8	Kegagalan saat unggah	Tidak normal	Proses unggah dihentikan sebelum	Sistem tidak mencatat objek sebagai berkas valid	Berhasil

Tabel 4.1 Hasil Pengujian

No	Parameter Pengujian	Kondisi	Skenario Pengujian	Hasil yang Diharapkan	Hasil Uji
			manifest dikomit		
9	Permintaan unduh objek tidak tersedia	Tidak normal	Pengguna meminta objek yang tidak memiliki manifest valid	Sistem menolak permintaan dan tidak mengembalikan data kosong sebagai objek sah	Berhasil
10	Simulasi manipulasi <i>ransomware</i>	Tidak normal	Penyerang menguasai lapisan API dan mencoba menghapus atau menimpa objek melalui jalur aplikasi	Sistem menolak operasi destruktif terhadap <i>Vault Core</i>	Berhasil

Berdasarkan rancangan pengujian pada Tabel 4.1, parameter normal digunakan untuk memastikan fungsi utama berjalan sesuai kebutuhan. Pengujian unggah dan unduh memvalidasi alur penyimpanan serta rekonstruksi objek. Pengujian berkas identik dan berkas dengan perubahan sebagian digunakan untuk menilai efektivitas deduplikasi melalui penggunaan ulang referensi *chunk*. Hasil pengujian menunjukkan bahwa sistem dapat menghindari penulisan ulang seluruh berkas ketika sebagian *chunk* sudah tersedia pada repositori.

Parameter tidak normal digunakan untuk mengevaluasi ketahanan sistem terhadap kegagalan proses dan masukan yang tidak sesuai. Sistem harus menolak berkas tidak valid, menjaga konsistensi saat unggah gagal, serta mencegah objek

yang belum lengkap tercatat sebagai data sah. Pengujian ini penting karena sistem penyimpanan cadangan harus tetap konsisten meskipun terjadi gangguan saat proses penyimpanan berlangsung.

Pengujian simulasi manipulasi *ransomware* menjadi bagian utama dalam pembahasan hasil. Skenario ini menempatkan lapisan API sebagai komponen yang telah dikuasai penyerang. Sistem diuji dengan permintaan penghapusan atau penimpaan objek melalui jalur aplikasi. Hasil yang diharapkan adalah *Vault Core* tetap menolak operasi destruktif dan menjaga *chunk* fisik serta manifest objek tetap tersedia. Hasil pengujian menunjukkan bahwa *Vault Core* menolak operasi destruktif dari lapisan API, sehingga *chunk* fisik dan manifest objek tetap tersedia meskipun lapisan aplikasi diasumsikan telah dikuasai penyerang.

Secara umum, pembahasan hasil menunjukkan bahwa prototipe dirancang untuk membedakan metadata logis pada aplikasi dan data permanen pada *Vault Core*. Perbandingan kondisi normal dan tidak normal diperlukan untuk membuktikan bahwa sistem tidak hanya dapat menyimpan dan mengambil berkas, tetapi juga mampu menjaga konsistensi serta membatasi manipulasi terhadap repositori cadangan.

4.3 Pengembangan Ke Tugas Akhir

Pengembangan pada tahap Tugas Akhir diarahkan untuk melanjutkan hasil Proyek Utama Informatika yang masih berfokus pada prototipe dasar *Immutable Object Storage* berbasis *Content-Addressable Storage* dan *Fast Content-Defined Chunking*. Pada tahap Proyek Utama Informatika, sistem difokuskan untuk membuktikan alur utama penyimpanan, deduplikasi *chunk*, *soft delete*, rekonstruksi objek, akses baca melalui *read-only mount*, serta pengujian konsistensi data dan simulasi manipulasi *ransomware*. Pengembangan ke tahap Tugas Akhir dapat difokuskan pada penambahan *Concurrent Garbage Collection* dan *Read-Proxy*. *Concurrent Garbage Collection* digunakan untuk membersihkan *chunk* yang tidak lagi memiliki referensi aktif tanpa mengganggu proses baca dan tulis yang sedang berjalan. *Read-Proxy* digunakan untuk menggantikan akses baca langsung ke

direktori penyimpanan fisik, sehingga proses rekonstruksi dan pengunduhan objek tetap dikendalikan melalui *Vault Core*.

BAB V

SIMPULAN

Berdasarkan model dan prototipe yang telah dihasilkan, penelitian menyimpulkan bahwa sistem *Immutable Object Storage* berbasis *Content-Addressable Storage* (CAS) dan *Fast Content-Defined Chunking* (FastCDC) dapat menjadi fondasi awal untuk melindungi repositori cadangan dari manipulasi *ransomware*. Sistem dirancang dengan pemisahan otoritas antara API Service dan *Vault Core*, sehingga lapisan aplikasi tidak memiliki kendali langsung terhadap *chunk* fisik, *manifest* objek, dan metadata permanen. Prototipe juga telah merepresentasikan alur utama penyimpanan, mulai dari unggah berkas, pemecahan berkas menjadi *chunk*, perhitungan *hash*, deduplikasi, pencatatan *manifest*, *soft delete* logis, hingga pengunduhan kembali objek. Antarmuka web berperan sebagai media demonstrasi untuk memperlihatkan alur penyimpanan dan metadata keamanan, sedangkan kontribusi utama penelitian berada pada mekanisme penyimpanan di *Vault Core*.

Hasil pengujian menunjukkan bahwa prototipe mampu memperlihatkan konsep penyimpanan *immutable* dan efisiensi kapasitas melalui penggunaan ulang *chunk* yang sama. Sistem juga mampu merekonstruksi objek berdasarkan *manifest*, menjaga pemisahan antara metadata logis aplikasi dan data permanen pada *Vault Core*, serta membatasi operasi destruktif melalui jalur aplikasi sesuai rancangan. Penelitian masih terbatas pada lingkungan *single-node* dan belum mencakup *garbage collection* fisik, *read-proxy*, *high availability*, serta replikasi terdistribusi. Keterbatasan tersebut menjadi dasar pengembangan pada tahap Tugas Akhir agar sistem lebih matang dari sisi keamanan, efisiensi kapasitas, dan kesiapan operasional.

DAFTAR PUSTAKA

- Banks, Alex., & Porcello, Eve. (2020). *Learning React : modern patterns for developing React apps*. 310. <https://www.oreilly.com/library/view/learning-react-2nd/9781492051718/>
- Bodner, Jon. (2021). *Learning Go : an idiomatic approach to real-world Go programming*.
- Buyya, R., Broberg, J., & Goscinski, A. (2011). Cloud Computing: Principles and Paradigms. *Cloud Computing: Principles and Paradigms*, 1–637. <https://doi.org/10.1002/9780470940105>
- Caporaso, P., Bianchi, G., & Quaglia, F. (2024). *VaultFS: Write-once Software Support at the File System Level Against Ransomware Attacks*.
- Cybersecurity and Infrastructure Security Agency, Federal Bureau of Investigation, Multi-State Information Sharing and Analysis Center, & Department of Health and Human Services. (2024). *#StopRansomware: RansomHub Ransomware*. <https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-242a>
- Cybersecurity and Infrastructure Security Agency, Multi-State Information Sharing and Analysis Center, National Security Agency, & Federal Bureau of Investigation. (2023). *#StopRansomware Guide*. <https://www.cisa.gov/stopransomware/ransomware-guide>
- Drake, Joshua., & Worsley, John. (2011). *Practical PostgreSQL*. 638. https://books.google.com/books/about/Practical_PostgreSQL.html?hl=id&id=52ENWgsOWLUC
- Higuchi, K., & Kobayashi, R. (2026). *Impact of File-Open Hook Points on Backup Ratio in ROFBS on XFS*. <http://arxiv.org/abs/2603.16364>
- Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, & Zooko Wilcox-O'Hearn. (2021). *BLAKE3 one function, fast everywhere*. <https://blake3.io>
- Jin, H., Zhang, C., Yu, H., Shi, S., Zhang, N., Hou, Y. T., & Lou, W. (2026). *Trusting What You Cannot See: Auditable Fine-Tuning and Inference for Proprietary AI*. <http://arxiv.org/abs/2603.07466>

- Kim, B. H., Hong, S. M., & Mannan, M. (2026). *Rhea: Detecting Privilege-Escalated Evasive Ransomware Attacks Using Format-Aware Validation in the Cloud*. <http://arxiv.org/abs/2601.18216>
- Kleppmann, Martin., & Riccomini Chris, . (2026). *Designing data-intensive applications : the big ideas behind reliable, scalable, and maintainable systems*.
- Lv, Y., Li, Q., Xu, Q., Gao, C., Yang, C., Wang, X., & Xue, C. J. (2025). *Rethinking LSM-tree based Key-Value Stores: A Survey*. <http://arxiv.org/abs/2507.09642>
- Múzquiz, G. G., González-Gómez, J., & Soriano-Salvador, E. (2025). *The Reverse File System: Towards open cost-effective secure WORM storage devices for logging*. <https://doi.org/10.1016/j.cose.2025.104786>
- Pipalani, Y., Raj, H., Ghosh, R., Bhargava, V., & Dutta, D. (2025). *Go-UT-Bench: A Fine-Tuning Dataset for LLM-Based Unit Test Generation in Go*. <http://arxiv.org/abs/2511.10868>
- Pressman, R. S. ., & Maxim, B. R. . (2020). *Software engineering : a practitioner's approach*. 671.
https://books.google.com/books/about/Software_Engineering.html?hl=id&id=taIKxAEACAAJ
- Rahman, A. N., Hantono, B. S., & Putra, G. D. (2025). *TruChain: A Multi-Layer Architecture for Trusted, Verifiable, and Immutable Open Banking Data*. <http://arxiv.org/abs/2507.08286>
- Silberschatz, Abraham., Galvin, P. B. ., & Gagne, Greg. (2019). *Operating system concepts*. 864.
- Sophos. (2024). *The impact of compromised backups on ransomware outcomes*. <https://www.sophos.com/en-us/blog/the-impact-of-compromised-backups-on-ransomware-outcomes>
- Stallings, W., Brown, L., Bauer, M. D., & Howard, M. (2023). *Computer security : principles and practice (5th edition)*. 336.
- Udayashankar, S., Baba, A., & Al-Kiswany, S. (2026). *Accelerating Data Chunking in Deduplication Systems using Vector Instructions*. <http://arxiv.org/abs/2508.05797>

- Vacca, John. (2020). *Cloud Computing Security, 2nd Edition* . 522.
- Xia, W., Zhou, Y., Jiang, H., Feng, D., Hua, Y., Hu, Y., Liu, Q., & Zhang, Y. (2016). {FastCDC}: A Fast and Efficient {Content-Defined} Chunking Approach for Data Deduplication. Dalam *2016 USENIX Annual Technical Conference (USENIX ATC 16)*.
<https://www.usenix.org/conference/atc16/technical-sessions/presentation/vesuna>
- Xu, Y., Sivaraman, P., Devarajan, H., Mohror, K., & Bhatele, A. (2024). *ML-based Modeling to Predict I/O Performance on Different Storage Sub-systems*.
<http://arxiv.org/abs/2312.06131>

LAMPIRAN